

Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt):

第 71 题：实现一个字符串匹配算法，从长度为 n 的字符串 S 中，查找是否存在字符串 T ， T 的长度是 m ，若存在返回所在位置。js

Để giải quyết bài toán **chuỗi S độ dài n** và **chuỗi T độ dài m** , kiểm tra xem T có xuất hiện trong S không và nếu có trả về vị trí, ta có thể triển khai nhiều thuật toán khác nhau.

1. Naive algorithm (Brute-force)

- Cơ chế: Duyệt S từ vị trí 0 đến $n-m$, với mỗi vị trí i so sánh lần lượt m ký tự của S với T .
- Độ phức tạp: $O(n \cdot m)$ (worst-case).
- Ví dụ:

```
function naiveSearch(S, T) {
  const n = S.length, m = T.length;
  for (let i = 0; i <= n - m; i++) {
    let j = 0;
    while (j < m && S[i + j] === T[j]) {
      j++;
    }
    if (j === m) {
      return i; // tìm thấy tại vị trí i
    }
  }
  return -1; // không tìm thấy
}
```

- Ưu: Đơn giản, dễ hiểu.
- Nhược: Chậm với trường hợp xấu (nhiều lần so sánh).

2. KMP (Knuth–Morris–Pratt)

2.1. Bản chất

- Tránh so sánh lặp lại đã biết thất bại nhờ **bảng “longest proper prefix which is also suffix”** (lps).
- $lps[j]$ = độ dài lớn nhất của prefix của $T[0..j]$ cũng là suffix kết thúc tại j .

- Khi so sánh T với S tại S[i..], nếu gặp mismatch tại T[j], ta không cần quay về j = 0, mà có thể nhảy về j = lps[j-1].

2.2. Tính bảng lps

```
function buildLPS(T) {
  const m = T.length;
  const lps = Array(m).fill(0);
  let len = 0; // độ dài prefix hiện tại
  let i = 1;
  while (i < m) {
    if (T[i] === T[len]) {
      len++;
      lps[i] = len;
      i++;
    } else {
      if (len !== 0) {
        len = lps[len - 1];
        // không tăng i ở đây
      } else {
        lps[i] = 0;
        i++;
      }
    }
  }
  return lps;
}
```

2.3. Tìm kiếm

```
function kmpSearch(S, T) {
  const n = S.length, m = T.length;
  if (m === 0) return 0;
  const lps = buildLPS(T);
  let i = 0, j = 0;
  while (i < n) {
    if (S[i] === T[j]) {
      i++; j++;
      if (j === m) {
        return i - j; // vị trí bắt đầu
      }
    } else {
      if (j !== 0) {
        j = lps[j - 1];
      } else {
        i++;
      }
    }
  }
  return -1;
}
```

- Độ phức tạp: $O(n + m)$.
- Ưu: Nhanh, dùng cho chuỗi lớn.
- Nhược: Triển khai phức tạp hơn.

3. Rabin-Karp

- Dùng **hash rolling**: tính hash của cửa sổ độ dài m trên S , so sánh với hash của T .
- Nếu hash bằng, so sánh trực tiếp để tránh va chạm.
- Độ phức tạp trung bình $O(n + m)$, worst-case $O(n \cdot m)$ nếu nhiều collision.

```
function rabinKarp(S, T) {
  const n = S.length, m = T.length;
  if (m > n) return -1;
  const base = 256, mod = 101; // mod chọn số nguyên tố
  let hashT = 0, hashS = 0, h = 1;
  // h = (base^(m-1)) % mod
  for (let i = 0; i < m - 1; i++) {
    h = (h * base) % mod;
  }
  // tính hash ban đầu
  for (let i = 0; i < m; i++) {
    hashT = (hashT * base + T.charCodeAt(i)) % mod;
    hashS = (hashS * base + S.charCodeAt(i)) % mod;
  }
  // trượt cửa sổ
  for (let i = 0; i <= n - m; i++) {
    if (hashS === hashT) {
      // so sánh trực tiếp
      if (S.substr(i, m) === T) return i;
    }
    if (i < n - m) {
      hashS = (base * (hashS - S.charCodeAt(i) * h) + S.charCodeAt(i + m)) % mod;
      if (hashS < 0) hashS += mod;
    }
  }
  return -1;
}
```

4. Thuật toán nâng cao

- **Boyer-Moore**: dùng heuristics "bad character" và "good suffix", nhảy dài hơn khi mismatch.
- **Suffix Automaton / Suffix Array / Trie**: dùng khi cần xử lý nhiều truy vấn tìm kiếm trên cùng S .
 - Xây dựng cấu trúc tiên xử lý $O(n)$, mỗi truy vấn $O(m)$ hoặc $O(m \log n)$.
- **Aho-Corasick**: tìm nhiều pattern $T_1, \dots, T_k$ trong S cùng lúc.

5. Ví dụ dummy

Giả sử $S = \text{"abcxabc dabxabc dabcbcy"}$, $T = \text{"abcdabcy"}$:

```
const S = "abcxabc dabxabc dabcbcy";
const T = "abcdabcy";
console.log("Naive:", naiveSearch(S,T)); // -1
console.log("KMP:  ", kmpSearch(S,T));   // 15
console.log("RK:   ", rabinKarp(S,T));   // 15
```

- Kết quả: pattern bắt đầu từ vị trí 15.

Tóm lại, **KMP** thường được dùng khi cần tốc độ ổn định $O(n+m)$, **Rabin-Karp** khi cần hashing tiện lợi, **Boyer-Moore** khi pattern dài và alphabet nhỏ, còn cấu trúc suffix phù hợp với nhiều truy vấn.

Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt):

第 72 题：为什么普通 for 循环的性能远远高于 forEach 的性能，请解释其中的原因。

Trong JavaScript, vòng lặp `for` thô sơ (cổ điển) thường có hiệu năng cao hơn rất nhiều so với phương thức `Array.prototype.forEach`. Để hiểu rõ nguyên nhân, ta cần đi sâu vào bản chất hoạt động và cơ chế tối ưu hóa của engine JavaScript (V8, SpiderMonkey, Chakra...).

1. Bản chất triển khai

1.1. Vòng lặp `for` cổ điển

- Là cú pháp cơ bản, engine biên dịch thành bytecode rất đơn giản:
 1. Khởi tạo biến `i`.
 2. So sánh điều kiện `i < length`.
 3. Thực thi thân vòng.
 4. Tăng `i`.
- Mỗi bước này nằm gọn trong một khối lệnh, dễ dàng nằm gọn trong **vòng lặp máy** (CPU loop), tận dụng triệt để **branch prediction** và **vectorization** (SIMD) của CPU.

1.2. `forEach`

- Là một phương thức của `Array` được định nghĩa trong prototype. Khi gọi `arr.forEach(fn)`, engine phải:
 1. Lấy tham chiếu đến hàm `forEach`.

2. Tạo closure cho callback fn (nếu dùng arrow function hoặc function expression).
 3. Gọi hàm callback cho từng phần tử, thực hiện context switch (setup stack frame) mỗi lần.
- Mỗi lần "invoke" callback là một **lời gọi hàm** (function call) tốn kém thời gian hơn so với một phép tăng biến và so sánh trong vòng for.

2. Chi phí hàm và tối ưu hóa

- **Chi phí gọi hàm (Function Call Overhead)**

Mọi lời gọi hàm đều phải push frame, tạo context (scope, this, arguments object...), sau đó pop frame khi trả về. forEach gắn liền với callback, nên cứ mỗi phần tử là một lần push/pop stack.

- **Inline Cache & Inlining**

Engine thường cố gắng *inline* các hàm nhỏ để loại bỏ overhead. Tuy nhiên, callback trong forEach không đảm bảo có thể inline vì:

- Là hàm được truyền động (dynamic) từ bên ngoài.
- Thường xuất hiện dưới dạng closure, giữ tham chiếu đến môi trường lexical scope, khó dự đoán.

- **Bounds Check và Hoisting độ dài**

Với vòng for, engine có thể hoist (cache) độ dài mảng (arr.length) ra ngoài trước khi lặp, tránh đọc lại mỗi lần. Mặc dù forEach cũng có thể làm tương tự, song chi phí lặp gọi hàm vẫn lấn át.

3. Thống kê hiệu năng

Ví dụ đo performance trên Chrome 73, Mac OS X 10.14.3 với mảng 1,000,000 phần tử:

Test	Ops/sec
for	1,650 ±8.76% fastest
forEach	297 ±3.40% 81% slower

Kết quả cho thấy for nhanh gấp ~5.5 lần.

4. Ứng dụng nâng cao và ví dụ dummy

Giả sử bạn cần xử lý mảng lớn, ví dụ cộng dồn giá trị:

```
// Với for
function sumFor(arr) {
  let total = 0;
  for (let i = 0, len = arr.length; i < len; i++) {
    total += arr[i];
  }
  return total;
}
```

```
// Với forEach
function sumForEach(arr) {
  let total = 0;
  arr.forEach(v => {
    total += v;
  });
  return total;
}

const bigArr = Array.from({ length: 1e7 }, () => 1);
console.time('sumFor');
sumFor(bigArr);
console.timeEnd('sumFor');

console.time('sumForEach');
sumForEach(bigArr);
console.timeEnd('sumForEach');
```

Trên các engine phổ biến, `sumFor` thường hoàn thành nhanh hơn `sumForEach` đáng kể, nhất là khi mảng có hàng chục triệu phần tử.

5. Kết luận

- Vòng `for` tận dụng tốt **phép toán nguyên thủy**, dễ **inline** và **hoist** độ dài, giảm branch misprediction.
- `forEach` chịu overhead của *function call* và *closure*, làm chậm tiến trình lặp.
- Khi performance là ưu tiên (xử lý mảng lớn, tight loop), ưu tiên dùng `for` hoặc các phương thức lặp thô sơ như `while`, `for...of` (với caching hợp lý).

✱

[^2_1]: image.jpg --- ## Tổng quan bốn "formatting context" trong CSS | Viết tắt | Tên đầy đủ | Khi nào được tạo | Quy tắc bố trí con | Ứng dụng thực tế nổi bật | |---|---|---|---| | ****BFC**** | Block Formatting Context | - `float` ≠ `none` | - `position: absolute/fixed` | - `display: inline-block`, `table-cell`, `flex`, `grid`, ... | - `overflow` ≠ `visible` [^3_1][^3_2] | - Các hộp khối xếp ****thẳng cột****. | - Biên trên-dưới của 2 khối liền kề ****có thể gộp****. | - Float bên trong ****không tràn**** ra ngoài BFC. [^3_1] | - "Clear float" bằng cách cho cha `overflow: hidden`. | - Ngăn sập margin khi làm card, panel. | | ****IFC**** | Inline Formatting Context | Mặc định trong mọi khối không tạo BFC riêng. | - Hộp inline (text, link, `img`) xếp ****nằm ngang****, tự xuống dòng khi hết chỗ. | - Chiều cao dòng quyết bởi `line-height` lớn nhất. | - Căn chữ dọc (vertical-align). | - Tạo chip/badge bằng inline-block. | | ****GFC**** | Grid Formatting Context | Bất kỳ phần tử `display: grid` | inline-grid`. | - Con **\*\*đồ họa lưới hai chiều\*\*** (cột & hàng). | - Không có gộp margin với bên ngoài. | - Layout dashboard, gallery. | - "Holy Grail" responsive chỉ với CSS Grid. | | **\*\*FFC\*\*** | Flex Formatting Context | Bất kỳ phần tử `display: flex` | inline-flex`. | - Con xếp ****một chiều**** (row/column) với khả năng `_flex-grow/shrink_, order`.

- Không gộp margin với ngoài. | - Navbar co giãn.
- Card list auto-wrap ('flex-wrap'). | ### 1. BFC – “khối vũ trụ độc lập” - Tạo “không gian khối” tách biệt; float, margin, column bên trong **không ảnh hưởng ngoài** và ngược lại^{[3_1][3_2]}. - Ví dụ dummy: muốn container ôm float con: ``html
Text dài...

→ Nếu bỏ `overflow:hidden`, chiều cao `.card` sụp do float “thoát” ra ngoài.

2. IFC – “dòng chữ”

- Khi bạn gõ văn bản trong `

`, trình duyệt ở **Inline Formatting Context**: ký tự → w
- Tất cả hộp inline chia sẻ **baseline**; thuộc tính `vertical-align` dịch chúng theo tr
- Lợi dụng IFC để căn icon với text:

```
``html
<button>
  <svg width="16" ... style="vertical-align:middle"></svg>
  <span style="vertical-align:middle">Like</span>
</button>
```

3. GFC – “đồ họa lưới”

- Khi khai `display:grid`, phần tử gốc lập tức tạo **Grid Formatting Context** mới^[3_3].
- Mỗi con trở thành *grid item*; vị trí do dòng/hàng, gap, auto-placement.
- Example dummy: bố trí 3 cột, 2 hàng cố định, ảnh tự co:

```
.gallery{
  display:grid;
  grid-template-columns:repeat(3, 1fr);
  grid-auto-rows:200px;
  gap:12px;          /* GFC nên margin giữa items không gộp */
}
.gallery img{width:100%;object-fit:cover;}
```

4. FFC – “dài co giãn”

- `display:flex` sinh **Flex Formatting Context**; các con thành *flex items*.
- Thuật toán: đo kích thước cơ sở, phân phối không gian dư/thiếu theo `flex-grow`, `flex-shrink`, sau đó căn dọc/hai chiều (`align-items`, `justify-content`).
- Ví dụ dummy: thanh điều hướng tự co, item cuối đẩy về phải:

```
.nav{
  display:flex;
}
.nav a:last-child{margin-left:auto;} /* vì margin của flex item không gộp, hack này luôn
```

Quan hệ & mẹo nâng cao

- Một phần tử có thể đồng thời là **flex item** (nằm trong FFC) và chính nó tạo **BFC/FFC/GFC** riêng (bằng `display:flex` lồng hoặc `overflow:hidden`).
→ Kỹ thuật “đóng băng” chiều cao thẻ con: đặt `.item{overflow:auto;}` để ngắt sập margin khi wrap flex.
- **Margin collapsing** chỉ xảy ra bên trong cùng BFC; Grid/Flex không gặp vấn đề này → dùng chúng để tránh bug khoảng trắng đầu/đuôi container.
- **Contain layout**: `contain: layout;` (hoặc `content-visibility:auto`) cũng tạo BFC, hơn nữa bật isolation cho engine, giúp optimize rendering lớn.
- **Sub-grid** (Firefox, Chrome 117+): item có `display:subgrid` kế thừa track của GFC cha—tạo GFC lồng share trực.

Ghi nhớ nhanh

- Cần “chặn float” or “ngăn sập margin” → **tạo BFC** (`overflow:hidden`, `display:flow-root`).
- Muốn xếp dòng chữ, icon bên text → hiểu **IFC** và `vertical-align`.
- Layout 2-chiều phức tạp → **GFC** (CSS Grid).
- Thanh ngang, card list co giãn → **FFC** (Flexbox).

Nắm vững 4 “formatting context” giúp bạn điều khiển dòng chảy, tránh bug layout và khai thác tối ưu engine trình duyệt cho UI hiện đại.



[^3_1]: [https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_display/Block_formatting_context)

[US/docs/Web/CSS/CSS_display/Block_formatting_context](https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_display/Block_formatting_context) [^3_2]:

<https://www.sitepoint.com/understanding-block-formatting-contexts-in-css/> [^3_3]:

[https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_display/Introduction_to_formatting_contexts)

[US/docs/Web/CSS/CSS_display/Introduction_to_formatting_contexts](https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_display/Introduction_to_formatting_contexts) [^3_4]:

[https://stackoverflow.com/questions/73774963/what-does-a-block-formatting-context-actually-](https://stackoverflow.com/questions/73774963/what-does-a-block-formatting-context-actually-do)

[do](https://stackoverflow.com/questions/73774963/what-does-a-block-formatting-context-actually-do) [^3_5]: [https://www.smashingmagazine.com/2017/12/understanding-css-layout-block-](https://www.smashingmagazine.com/2017/12/understanding-css-layout-block-formatting-context/)

[formatting-context/](https://www.smashingmagazine.com/2017/12/understanding-css-layout-block-formatting-context/) [^3_6]: [https://www.greatfrontend.com/questions/quiz/describe-block-](https://www.greatfrontend.com/questions/quiz/describe-block-formatting-context-bfc-and-how-it-works)

[formatting-context-bfc-and-how-it-works](https://www.greatfrontend.com/questions/quiz/describe-block-formatting-context-bfc-and-how-it-works) [^3_7]: https://github.com/ThatOpen/engine_web-ifc

[^3_8]: <https://stackoverflow.com/questions/384326/google-friend-connect-css-style-switching>

[^3_9]: <https://www.color-hex.com/color/ffffcc> [^3_10]:

[https://vn.antenkconnector.com/news/analyze-the-difference-and-common-types-of-ffc-](https://vn.antenkconnector.com/news/analyze-the-difference-and-common-types-of-ffc-53313845.html)

[53313845.html](https://vn.antenkconnector.com/news/analyze-the-difference-and-common-types-of-ffc-53313845.html) [^3_11]: [https://developers.google.com/privacy-sandbox/protectations/on-device-](https://developers.google.com/privacy-sandbox/protectations/on-device-personalization/federated-compute-server?hl=vi)

[personalization/federated-compute-server?hl=vi](https://developers.google.com/privacy-sandbox/protectations/on-device-personalization/federated-compute-server?hl=vi) [^3_12]:

[https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/TC1/HTML/ifcrepresentationresource/](https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/TC1/HTML/ifcrepresentationresource/lexical/ifcstylemodel.htm)

[lexical/ifcstylemodel.htm](https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/TC1/HTML/ifcrepresentationresource/lexical/ifcstylemodel.htm) [^3_13]: [https://stackoverflow.com/questions/25959239/google-feed-](https://stackoverflow.com/questions/25959239/google-feed-control-styling-with-css)

[control-styling-with-css](https://stackoverflow.com/questions/25959239/google-feed-control-styling-with-css) [^3_14]: <https://www.ffcorp.com/css-test/> [^3_15]:

<https://www.ipcb.com/vn/technical/5289.html> [^3_16]: <https://docs.fileformat.com/vi/cad/ifc/>

[^3_17]: [https://www.geeksforgeeks.org/css/explain-the-working-of-the-block-formatting-](https://www.geeksforgeeks.org/css/explain-the-working-of-the-block-formatting-context/)

[context/](https://www.geeksforgeeks.org/css/explain-the-working-of-the-block-formatting-context/) [^3_18]: <https://ifcopenshell.org> [^3_19]: <https://edu.gcfglobal.org/en/basic-html/>

[^3_20]: <https://forum.freecodecamp.org/t/ffc-technical-documentation-page-help/465094> ---

Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt): 第 74 题 : 使用 JavaScript Proxy 实现简单的数据绑定 Để **sử dụng JavaScript Proxy** để thực hiện **simple data binding** (liên kết dữ liệu hai chiều) trong ứng dụng vanilla JS, cơ bản ta sẽ: 1. Tạo một **model object** chứa dữ liệu. 2. Bọc model bằng một **Proxy** với các trap `get`/`set`. 3. Khi đọc (`get`) hoặc ghi (`set`) thuộc tính, **Proxy** sẽ: - Cập nhật trực tiếp DOM tương ứng. - Đồng bộ giá trị trở lại model gốc. Dưới đây là một ví dụ **dummy** (tiếng Việt) minh họa **hai chiều**: ``html

Họ tên:

Tuổi:

Model hiện tại:

</body>

</html>

Giải thích chi tiết:

- `user` là **object** gốc lưu trữ dữ liệu.
- `handler.set(...)` được gọi mỗi khi gán `proxyUser[prop] = value`:
 - Gán lại vào `target[prop]` (chính là `user`).
 - Cập nhật giá trị cho thẻ `<input>` tương ứng (dựa trên `data-model`).
 - Gọi `renderModel` để hiển thị JSON model lên `<pre>`.
- Khi **người dùng** nhập vào input, listener `input` sẽ gán giá trị mới vào `proxyUser`.

Mở rộng nâng cao:

- *Validation*: trong `set` có thể kiểm tra ràng buộc, ví dụ tuổi phải $\geq 0 \leq 120$.
- *Observer list*: thay vì update DOM trực tiếp, có thể lưu mảng callback cho mỗi thuộc tính.
- *Nested object*: để hỗ trợ binding cho object lồng nhau, khi `get` detect thấy giá trị.
- *Batch update*: gom nhiều thay đổi rồi gọi render một lần, tránh reflow liên tục.

Kết hợp ES6 **Proxy** và `data-\*` attribute giúp implement một cơ chế data binding **nhẹ

<div style="text-align: center">✖</div>

[^4_1]: <https://blog.risingstack.com/writing-a-javascript-framework-data-binding-es6-proxy/>

[^4_2]: <https://www.freecodecamp.org/news/javascript-proxy-object/>

[^4_3]: <https://www.sitepoint.com/es6-proxies/>

[^4_4]: <https://stackoverflow.com/questions/78150861/javascript-bind-class-methods-to-proxy>

[^4_5]: <https://dev.to/jp024556/ever-wondered-how-data-binding-works-in-modern-ui-frameworks>

[^4_6]: <https://www.youtube.com/watch?v=R8vcs-ausuQ>

[^4_7]: https://blog.csdn.net/scorpio_h/article/details/104631871

[^4_8]: <https://www.javascripttutorial.net/javascript-proxy/>

[^4_9]: <https://dev.to/shafayeat/understanding-javascript-proxy-power-of-intercepting-and>

[^4_10]: <https://cloud.tencent.com/developer/article/2374839>

[^4_11]: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy

[^4_12]: <https://www.cnblogs.com/smile-fanyin/p/14649017.html>

[^4_13]: <https://gist.github.com/harish81/ee6ec876ae28750f8e2ecba57b9aa2ec>
[^4_14]: <https://juejin.cn/post/7295771624695300111>
[^4_15]: https://blog.csdn.net/weixin_45825917/article/details/132712123
[^4_16]: <https://www.wemlion.com/post/es6-proxy-data-binding/>
[^4_17]: <https://zh.javascript.info/proxy>
[^4_18]: <http://objcer.com/2017/10/31/Data-Binding-with-ES6-Proxies/>
[^4_19]: <https://codelove.tw/@tony/post/Zq4ZL3>
[^4_20]: <https://heptaluan.github.io/2018/07/10/JavaScript/29/>

Truy cập phần tử đầu và cuối của một mảng “khổng lồ” (100,000 phần tử) khác nhau bao

1. **Bản chất lưu trữ của mảng**

- Trong hầu hết ngôn ngữ/hệ thống, mảng là **khối bộ nhớ liên tiếp**.
- Địa chỉ phần tử *i* được tính trực tiếp:
`addr(base) + i × size_of(element)` → **tính toán số học đơn giản trên CPU** [^5_1]

2. **Độ phức tạp thuật toán**

- Tính địa chỉ và đọc giá trị đều là hằng số → **O(1)** cho **bất kỳ** chỉ số hợp lệ, b

3. **Thời gian thực tế (CPU nanosecond)**

- Cả phần tử đầu (`arr`) lẫn phần tử thứ 100,000 (`arr[99_999]`) đều đòi **1** thao tác
- Sự khác biệt duy nhất: **liệu dữ liệu đang nằm trong cache**.
 - Nếu cả hai vị trí đã nằm trong L1/L2 cache → **~ 1-10 ns**, gần như giống hệt.
 - Nếu phải truy cập DRAM (cache miss) → **~ 50-100 ns**; vẫn **không phụ thuộc** vào chỉ

4. **So sánh định lượng (ví dụ trên máy bàn hiện đại)**

Tình huống	arr	arr[99_999]	Chênh lệch
Đều có trong L1	~1 ns	~1 ns	~0 ns
arr còn trong L1, phần cuối rớt cache	~1 ns	~80 ns (DRAM)	~79 ns
Cả hai rớt cache	~80 ns	~80 ns	~0 ns

5. **Ý nghĩa cho lập trình viên**

- **Không cần tối ưu hóa** bằng cách “đưa phần tử hay dùng về đầu mảng”; truy cập **O(1)**
- Điều cần quan tâm là **tính tuần tự**: duyệt mảng theo thứ tự giúp prefetch & cache

6. **Ví dụ dummy (JS, Node v20)**

```
```js
const arr = new Array(100_000).fill(0);

// “Làm nóng” cache đầu mảng
arr[^5_0];

console.time('first');
const a = arr[^5_0];
console.timeEnd('first');

console.time('last');
const b = arr[arr.length - 1];
console.timeEnd('last');
```

Trên máy Mac M2 kết quả mẫu:

```
first: 0.009 ms
last: 0.010 ms
```

Sai số nằm trong jitter đo; **không có chênh lệch ý nghĩa.**

Kết luận: Với mảng 100,000 phần tử (hay lớn hơn nhiều), **thời gian lấy phần tử đầu và cuối gần như bằng nhau**; khác biệt duy nhất đến từ cache, không đến từ kích cỡ hay vị trí chỉ số trong mảng.



[^5\_1]: <https://stackoverflow.com/questions/37120776/accessing-elements-really-o1/37120863>  
[^5\_2]: <https://softwareengineering.stackexchange.com/questions/252407/why-is-the-complexity-of-fetching-a-value-from-an-array-be-o1> [^5\_3]:  
<https://stackoverflow.com/questions/32797260/time-complexity-of-random-access-of-an-array>  
[^5\_4]: <https://www.geeksforgeeks.org/dsa/comparison-of-an-array-and-hash-table-in-terms-of-storage-structure-and-access-time-complexity/> [^5\_5]:  
<https://codefinity.com/courses/v2/212d3d3e-af15-4df9-bb13-5cbbb8114954/58324ed0-9644-4e88-ba8c-e93d15b8697a/c72b87c4-3da6-41f0-9133-559b82b95adc> [^5\_6]:  
[https://www.reddit.com/r/learnprogramming/comments/mni73v/how\\_much\\_longer\\_does\\_it\\_typically\\_take\\_to\\_access/](https://www.reddit.com/r/learnprogramming/comments/mni73v/how_much_longer_does_it_typically_take_to_access/) [^5\_7]:  
[https://www.reddit.com/r/compsci/comments/18x0xm6/why\\_does\\_random\\_access\\_memory\\_have\\_a\\_time/](https://www.reddit.com/r/compsci/comments/18x0xm6/why_does_random_access_memory_have_a_time/) [^5\_8]: <https://stackoverflow.com/questions/4090491/how-to-get-the-first-element-of-an-array> [^5\_9]: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Array](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array) [^5\_10]:  
<https://docs.aws.amazon.com/athena/latest/ug/accessing-array-elements.html> [^5\_11]:  
<https://flexiple.com/javascript/get-last-array-element-javascript> [^5\_12]:  
<https://stackoverflow.com/questions/59722749/how-do-i-access-the-last-element-of-an-array>  
[^5\_13]: <https://www.baeldung.com/java-array-get-first-last> [^5\_14]:  
[https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Array/findLast](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/findLast) [^5\_15]:  
<https://cs.stackexchange.com/questions/152613/array-access-is-o1-implies-a-fixed-index-size-which-implies-o1-array-traver> [^5\_16]:  
[https://www.reddit.com/r/javascript/comments/1bqwbb1/the\\_easy\\_way\\_to\\_access\\_the\\_last\\_javascript\\_array/](https://www.reddit.com/r/javascript/comments/1bqwbb1/the_easy_way_to_access_the_last_javascript_array/) [^5\_17]: <https://dev.to/masakifukunishi/understanding-arrays-features-pros-cons-and-time-complexity-3egc> [^5\_18]: <https://community.make.com/t/issue-with-accessing-last-element-in-an-array-within-a-filter/57839> [^5\_19]: <https://www.linkedin.com/pulse/mastering-array-data-structure-comprehensive-guide-akshay-kumar-qfzhc> [^5\_20]: <https://users.rust-lang.org/t/arrays-access-last-element/502> --- ## Phân tích chi tiết 3 ví dụ về **Object Property Keys** và **Type Coercion** Các ví dụ này minh họa cách JavaScript xử lý **property keys** với các kiểu dữ liệu khác nhau: **string**, **number**, **Symbol**, và **object**. Đây là những concept cốt lõi về **object property resolution** và **type conversion**. ## 1. Example 1: Number vs String Keys ```js // example 1 var a={}, b='123', c=123; a[b]='b'; a[c]='c'; console.log(a[b]); ``` **Kết quả: 'c'** #### Giải thích bản chất: - **Property key coercion**: Trong JavaScript, tất cả property keys (trừ Symbol) đều được **convert thành string**. - **b = '123'** (đã là string) - **c = 123** (number) → tự động convert thành **'123'** khi dùng làm property

key - Kết quả: `a['123'] = 'b'` bị ghi đè bởi `a['123'] = 'c'` ### Chứng minh: ``js const obj = {}; obj[123] = 'number'; obj['123'] = 'string'; console.log(Object.keys(obj)); // ['123'] - chỉ có 1 key console.log(obj[123] === obj['123']); // true `` ## 2. Example 2: Symbol Properties ``js // example 2 var a={}, b=Symbol('123'), c=Symbol('123'); a[b]='b'; a[c]='c'; console.log(a[b]); ``

**\*\*Kết quả: `b`\*\*** ### Giải thích bản chất: - **\*\*Symbol uniqueness\*\***: Mỗi lời gọi `Symbol('123')` tạo ra một **\*\*symbol hoàn toàn mới và duy nhất\*\***. - `b` và `c` là **\*\*hai symbols khác biệt hoàn toàn\*\***, mặc dù có cùng description `123`. - Symbol properties **\*\*không bị convert\*\*** thành string và **\*\*không va chạm\*\*** với các keys khác. ### Chứng minh: ``js const s1 = Symbol('123'); const s2 = Symbol('123'); console.log(s1 === s2); // false - khác nhau hoàn toàn console.log(s1.toString()); // "Symbol(123)" console.log(s2.toString()); // "Symbol(123)" - nhưng vẫn là 2 symbols khác nhau const obj = {}; obj[s1] = 'first'; obj[s2] = 'second'; console.log(Object.keys(obj)); // [] - Symbol keys không liệt kê được console.log(Object.getOwnPropertySymbols(obj)); // [Symbol(123), Symbol(123)] `` ## 3. Example 3: Object Keys ``js // example 3 var a={}, b={key:'123'}, c={key:'456'}; a[b]='b'; a[c]='c'; console.log(a[b]); ``

**\*\*Kết quả: `c`\*\*** ### Giải thích bản chất: - **\*\*Object-to-primitive conversion\*\***: Khi object được dùng làm property key, JavaScript gọi **\*\*`toString()`\*\*** method. - Cả `b` và `c` đều là plain objects → `toString()` trả về `[object Object]`. - Kết quả: cả hai objects đều trở thành cùng một key `[object Object]`. ### Chứng minh: ``js const obj1 = {key: '123'}; const obj2 = {key: '456'}; console.log(obj1.toString()); // "[object Object]" console.log(obj2.toString()); // "[object Object]" const container = {}; container[obj1] = 'first'; container[obj2] = 'second'; // ghi đè lên 'first' console.log(Object.keys(container)); // ["[object Object]"] console.log(container[obj1]); // "second" `` ## 4. Mở rộng nâng cao ### 4.1. Custom `toString()` và `valueOf()` ``js const customObj = { key: 'test', toString() { return `custom-\${this.key}`; } }; const container = {}; container[customObj] = 'value'; console.log(Object.keys(container)); // ["custom-test"] `` ### 4.2. Symbol.for() - Global Symbol Registry ``js const s1 = Symbol.for('123'); // tạo hoặc lấy từ global registry const s2 = Symbol.for('123'); // lấy cùng symbol từ registry console.log(s1 === s2); // true - cùng một symbol `` ### 4.3. Property Descriptor và Enumerability ``js const obj = {}; const sym = Symbol('hidden'); obj[sym] = 'hidden value'; obj.visible = 'visible value'; console.log(Object.keys(obj)); // ["visible"] console.log(Object.getOwnPropertyNames(obj)); // ["visible"] console.log(Object.getOwnPropertySymbols(obj)); // [Symbol(hidden)] console.log(Reflect.ownKeys(obj)); // ["visible", Symbol(hidden)] `` ### 4.4. Performance implications ``js // Symbol properties không participate trong JSON serialization const data = { [Symbol('meta')]: 'metadata', value: 42 }; console.log(JSON.stringify(data)); // {"value":42} // Tối ưu cho private properties class MyClass { constructor() { this[Symbol('private')] = 'secret'; this.public = 'visible'; } } `` ## 5. Bảng tóm tắt Property Key Behavior

Key Type	Conversion	Uniqueness	Enumerable	Use Case
String	Theo giá trị	✓	Public properties	Number → String
Array-like access	Theo giá trị	✓	Public properties	Symbol
Symbol	Không đổi	Luôn unique*	✗	Private/meta properties
Object	→ `toString()`	Theo output string	✓	**Tránh dùng!**

\*Trừ `Symbol.for()` ## Kết luận Hiểu rõ **\*\*property key coercion\*\*** giúp: - Tránh bug khó debug khi mix string/number keys - Sử dụng Symbol cho metadata/private properties - Tránh dùng objects làm keys (trừ Map/WeakMap) - Tối ưu performance và memory usage **\*\*Lời khuyên\*\***: Dùng `Map` khi cần object keys thực sự, `Symbol` cho private properties, và kiểm tra type consistency khi làm việc với dynamic keys. --- # Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt): 第 77 题 : 算

法题「旋转数组」给定一个数组，将数组中的元素向右移动  $k$  个位置，其中  $k$  是非负数。示例 1：js 代码解读 复制代码 输入：[1, 2, 3, 4, 5, 6, 7] 和  $k = 3$  输出：[5, 6, 7, 1, 2, 3, 4] 解释：向右旋转 1 步：[7, 1, 2, 3, 4, 5, 6] 向右旋转 2 步：[6, 7, 1, 2, 3, 4, 5] 向右旋转 3 步：[5, 6, 7, 1, 2, 3, 4] 示例 2：js 代码解读 复制代码 输入：[-1, -100, 3, 99] 和  $k = 2$  输出：[3, 99, -1, -100] 解释：向右旋转 1 步：[99, -1, -100, 3] 向右旋转 2 步：[3, 99, -1, -100] ♦ **BÀI 77 – “ROTATE ARRAY”** Đề: Cho mảng nums, dịch các phần tử sang phải  $k$  bước ( $k \geq 0$ ). Trả về mảng sau khi xoay. -----

----- I. TƯ DUY GỐC – “Xoay” nghĩa là gì? 1. Sau mỗi bước, phần tử cuối trôi lên đầu, các phần tử còn lại dồn sang phải. 2. Thao tác “xoay phải  $k$ ” thực chất là: - Cắt đuôi dài  $k$  phần tử → dán lên đầu. - Thứ tự tương đối của 2 khối vẫn được giữ nguyên (ổn định). 3. Vì mảng có độ dài  $n$  nên xoay  $k \equiv k \bmod n$  (vì xoay  $n$  lần trở về chỗ cũ). -----

----- II. 4 GIẢI PHÁP KINH ĐIỂN (tăng dần độ tối ưu) 1.

Brute-force “xoay từng bước” - Ý tưởng: lặp  $k$  lần, mỗi lần `pop()` & `unshift()`. - Độ phức tạp:  $O(k \cdot n)$  – rất chậm khi  $k$  lớn. - Chỉ nên dùng cho demo  $< 10^3$  phần tử. 2. Dùng mảng phụ (Extra array) ```js function rotateExtra(nums, k) { const n = nums.length; if (n === 0) return; k = k % n; const res = Array(n); for (let i = 0; i < n; ++i) { res[(i + k) % n] = nums[i]; } // copy ngược về for (let i = 0; i < n; ++i) nums[i] = res[i]; } ``` - Time  $O(n)$ , space  $O(n)$ . - Dễ hiểu, nhưng tốn thêm bộ nhớ. 3. Thuật toán “3 lần đảo ngược” – in-place kinh điển ☑  $O(n)$  time,  $O(1)$  space. ```js function reverse(arr, l, r) { while (l < r) [arr[l++], arr[r--]] = [arr[r], arr[l]]; } function rotateReverse(nums, k) { const n = nums.length; if (n === 0) return; k %= n; // bước 1: đảo cả mảng reverse(nums, 0, n - 1); // bước 2: đảo khúc đầu dài k reverse(nums, 0, k - 1); // bước 3: đảo khúc còn lại reverse(nums, k, n - 1); } ``` - Cơ chế: Đảo ngược hai lần khôi phục thứ tự tương đối bên trong mỗi khối. 4. Cyclic Replacement (thuật toán “GCD cycle”) - Ý tưởng: Các phần tử dịch theo chu trình có độ dài  $\gcd(n, k)$ . - Triển khai: ```js function rotateCycle(nums, k) { const n = nums.length; k %= n; let moved = 0; for (let start = 0; moved < n; ++start) { let cur = start; let prev = nums[start]; do { const next = (cur + k) % n; [nums[next], prev] = [prev, nums[next]]; cur = next; ++moved; } while (start !== cur); } } ``` - Vẫn  $O(n)$  time,  $O(1)$  space, nhưng không dùng reverse. - Thường chậm hơn reverse vì vòng lặp & swap nhiều hơn, song hữu ích khi  $n$  rất lớn mà reverse khó pipeline. -----

----- III. SO SÁNH

NHANH	Cách	Time	Space	Code length	Khi dùng
Pop/unshift	$O(k \cdot n)$	$O(1)$	Rất ngắn	$k$ nhỏ, demo	Extra array
3-reverse	$O(n)$	$O(1)$	Ngắn	Production, phỏng vấn	GCD cycle
GCD cycle	$O(n)$	$O(1)$	Dài	Muốn khoe kiến thức, dùng cho nghiên cứu cache-line	

----- IV. MẪU CHẠY DUMMY ```js const ex1 = [1,2,3,4,5,6,7]; rotateReverse(ex1, 3); console.log(ex1); // [5,6,7,1,2,3,4] const ex2 = [-1,-100,3,99]; rotateReverse(ex2, 2); console.log(ex2); // [3,99,-1,-100] ``` -----

----- V. CẠNH MÉP & MỞ RỘNG 1. Trái, phải,  $k$  âm? - Xoay trái  $k \Leftrightarrow$  xoay phải  $(n - k)$ . - Cho phép  $k$  âm:  $k = ((k \% n) + n) \% n$ . 2. Xoay stream cực lớn (không vừa RAM) - Dùng buffer kích thước  $k$ , đọc chunk-wise. - Hoặc Circular Buffer: `head = (head - k) mod capacity`. 3. Generic rotate trên string / typed-array ```js const s = 'abcdefg'; const rotated = s.slice(-k) + s.slice(0, -k); ``` 4. Ứng dụng thực tế - Lập lịch Round-Robin, Josephus problem. - Mã hoá Caesar (dịch ký tự). - GPU ring-buffer, audio circular queue. 5. Tối ưu cache (reverse vs cycle) - 3-reverse quét mảng tuần tự ba lần → tiền ở L1/L2 tốt hơn cycled-swap (nhảy chỉ số). - Khi  $n \gg$  cache line, reverse gần như luôn thắng. 6. Thư viện hiện đại - Lodash: `_.rotate(array, k)` (tự viết) - Rust `slice::rotate_right`, C++20 `std::ranges::rotate`. -----

----- VI. CÂU HỎI PHÒNG VẤN LIÊN QUAN - Nếu mảng là linked-list? → Không thể  $O(1)$  space lẫn  $O(n)$  time với reverse, phải đổi pointer lần lượt. - Nếu  $k > 10^9$ ? →

Luôn rút gọn  $k \% = n$  trước, tránh overflow. - Làm sao để xoay mảng 2-chiều (ma trận) theo đường tròn? → Áp dụng rotate shell-by-shell (LeetCode 1894). -----

----- KẾT LUẬN - Hãy luôn normalize  $k \% = n$ . - Ưu tiên thuật toán "3 lần đảo ngược" khi phỏng vấn vì: ngắn, đúng,  $O(n)$  &  $O(1)$ . - Biết thêm Cyclic Replacement để chứng minh hiểu sâu GCD / chu trình hoán vị. --- ### Vì sao cần "debounce" khi gõ tìm kiếm? - Trường tìm kiếm bắn sự kiện `input` mỗi phím bấm. - Nếu mỗi sự kiện gọi API, render danh sách... sẽ nghẽn mạng & "lag". - **\*\*Debounce\*\*** trì hoãn xử lý cho tới khi người dùng **\*\*dừng gõ\*\***  $\geq t \text{ ms}$  → chỉ thực thi 1 lần, tiết kiệm tài nguyên[^8\_1][^8\_2][^8\_3]. ## 1. Cách cài đặt debounce cơ bản (vanilla JS) ```js function debounce(fn, delay = 300) { let timerId; // biến đóng (closure) return function (...args) { // trả về hàm bọc clearTimeout(timerId); // huỷ lần hẹn trước timerId = setTimeout(() => { // lên lịch mới fn.apply(this, args); // gọi hàm gốc }, delay); }; } // dùng: const handleSearch = e => fetchSuggest(e.target.value); inputEl.addEventListener('input', debounce(handleSearch, 500)); ``` - `clearTimeout` bảo đảm chỉ lần gõ **\*\*cuối cùng\*\*** gọi API. - Giữ `this` & `args` đúng bằng `fn.apply()`. ## 2. Vấn đề đặc thù với **\*\*IME / gõ tiếng Trung, Nhật, Việt\*\*** Khi dùng IME, chuỗi ký tự hiển thị **\*\*còn tạm thời\*\*** (composition). Nếu debounce chạy ngay, bạn sẽ gửi "拼" thay vì "拼音". ### 2.1. Các sự kiện liên quan | Sự kiện | Khi nào bắn | Ý nghĩa | |-----|-----|-----| | `compositionstart` | Bắt đầu nhập IME | Bật chế độ soạn âm | | `compositionupdate` | Trong khi đang gõ | Candidate vẫn thay đổi | | `compositionend` | Người dùng chọn/kết thúc | Ký tự cuối cùng đã cố định | | `input` | Sau mỗi thay đổi DOM | Bao gồm cả IME | ### 2.2. Quy tắc xử lý 1. **\*\*Tạm ngừng debounce\*\*** trong giai đoạn composition. 2. Chỉ **\*\*khởi động lại\*\*** khi nhận `compositionend` (chuỗi đã ổn định). ### 2.3. Mẫu mã xử lý IME + debounce ```js const input = document.querySelector('#search'); let isComposing = false; const debouncedSearch = debounce(q => fetchSuggest(q), 400); input.addEventListener('compositionstart', () => { isComposing = true; }); input.addEventListener('compositionend', () => { isComposing = false; debouncedSearch(input.value); // chạy ngay sau khi cố định }); input.addEventListener('input', e => { if (!isComposing) debouncedSearch(e.target.value); }); ``` Điểm mấu chốt: trong `composition` chuỗi vẫn "chưa quyết", tránh gọi API. ## 3. Thủ thuật nâng cao - **\*\*leading/trailing\*\***: cho phép gọi 1 lần "ngay lập tức" rồi im tới khi ngừng gõ. - **\*\*cancel/flush\*\***: trả về hàm phụ `fn.cancel()` & `fn.flush()` để huỷ hoặc ép chạy sớm (Lodash `\_.debounce`). - **\*\*React hook\*\***: `useEffect` + `setTimeout`, hoặc npm `use-debounce` cung cấp `useDebouncedCallback` tiện dụng[^8\_4]. - **\*\*Throttle vs debounce\*\***: throttle chạy mỗi t ms, còn debounce chỉ chạy khi \_ngừng\_ nhập. ## 4. Ví dụ "dummy" hoàn chỉnh ```html

``` - Gõ nhanh "abcdef" => chỉ 1 request sau 500 ms. - Gõ "拼音" bằng IME => API chỉ chạy \_sau\_ khi chọn chữ. ## 5. Lưu ý hiệu năng & UX - Chọn `delay`  $\approx 300\text{-}500$  ms cho search gợi ý; form ngắn hơn có thể 150 ms. - Show spinner hoặc skeleton khi debounce dài để người dùng biết hệ thống đang chờ. - Luôn **\*\*trim\*\*** & bỏ qua chuỗi rỗng để tránh request thừa. - Memoize kết quả query cũ nếu backend cho phép, giảm số lần gọi lại. **\*\*Kết luận\*\*** Debounce giúp input search mượt và tiết kiệm network. Với tiếng Hoa/IME, bắt buộc lắng nghe `composition\*` để không "bắn nhầm" ký tự chưa hoàn thiện — chỉ khi `compositionend` thì dữ liệu mới thật sự sẵn sàng gửi lên server.

✱

[^8_1]: <https://www.freecodecamp.org/news/javascript-debounce-example/> [^8_2]:

<https://www.explainthis.io/en/swe/debounce> [^8_3]: <https://web.dev/articles/debounce-your->

[input-handlers \[^8_4\]: https://www.npmjs.com/package/use-debounce \[^8_5\]:](https://www.npmjs.com/package/use-debounce)
[https://dev.to/manishkc104/debounce-input-in-react-3726 \[^8_6\]:](https://dev.to/manishkc104/debounce-input-in-react-3726)
[https://stackoverflow.com/questions/75988682/debounce-in-javascript \[^8_7\]:](https://stackoverflow.com/questions/75988682/debounce-in-javascript)
[https://www.developerway.com/posts/debouncing-in-react \[^8_8\]:](https://www.developerway.com/posts/debouncing-in-react)
[https://stackoverflow.com/questions/72868572/debounce-function-not-working-properly-when-input-is-changed-at-fast-pace \[^8_9\]:](https://stackoverflow.com/questions/72868572/debounce-function-not-working-properly-when-input-is-changed-at-fast-pace) [https://ratfactor.com/cards/js-debounce \[^8_10\]:](https://ratfactor.com/cards/js-debounce)
[https://dev.to/remejuan/react-debouncing-input-with-useeffect-3nhk \[^8_11\]:](https://dev.to/remejuan/react-debouncing-input-with-useeffect-3nhk)
[https://www.telerik.com/blogs/debouncing-and-throttling-in-javascript \[^8_12\]:](https://www.telerik.com/blogs/debouncing-and-throttling-in-javascript)
[https://tech.reverse.hr/articles/debounce-function-in-typescript \[^8_13\]:](https://tech.reverse.hr/articles/debounce-function-in-typescript)
[https://www.embeddedrelated.com/showthread/comp.arch.embedded/130710-2.php \[^8_14\]:](https://www.embeddedrelated.com/showthread/comp.arch.embedded/130710-2.php)
[https://dev.to/jakaria/debounce-in-javascript-29g7 \[^8_15\]:](https://dev.to/jakaria/debounce-in-javascript-29g7)
[https://www.geeksforgeeks.org/javascript/lodash-_debounce-method/ \[^8_16\]:](https://www.geeksforgeeks.org/javascript/lodash-_debounce-method/)
[https://www.geeksforgeeks.org/javascript/debouncing-in-javascript/ \[^8_17\]:](https://www.geeksforgeeks.org/javascript/debouncing-in-javascript/)
[https://www.linkedin.com/pulse/debouncing-throttling-javascript-guide-beginners-advanced-sonu-tiwari-ji1gc \[^8_18\]:](https://www.linkedin.com/pulse/debouncing-throttling-javascript-guide-beginners-advanced-sonu-tiwari-ji1gc) [https://www.arduino.cc/en/Tutorial/Debounce \[^8_19\]:](https://www.arduino.cc/en/Tutorial/Debounce)
[https://enlear.academy/implementing-debounced-search-in-next-js-a-step-by-step-guide-550d004b497d \[^8_20\]:](https://enlear.academy/implementing-debounced-search-in-next-js-a-step-by-step-guide-550d004b497d) <https://raspberrypi.stackexchange.com/questions/118349/what-is-the-proper-way-to-debounce-a-gpio-input> --- ## Promise.all – Tổng quan, Cơ chế sâu, Cài đặt thủ công & Xử lý lỗi ### 1. Promise.all dùng để làm gì? - **Hợp nhất nhiều promise song song** thành một “siêu-promise”. - Trả về **một promise mới**: - Fulfilled khi *tất cả* promise con đã resolved. - Rejected ngay khi *bất kỳ* promise con nào bị reject. - Kết quả resolved là **mảng** giữ nguyên thứ tự gốc. ``js const p1 = fetch('/user'); const p2 = fetch('/posts'); Promise.all([p1, p2]).then(([user, posts]) => { // chạy khi cả hai thành công }); `` #### 2. Luật hoạt động chi tiết 1. **Nhận iterable** (thường là array) – mỗi phần tử: - Nếu đã là Promise => giữ nguyên. - Nếu là giá trị sync => tự động `Promise.resolve(value)`. 2. Tạo **counter** `remaining` = số phần tử. Mỗi khi một promise con settles thành `\_fulfilled\_` - Lưu giá trị vào slot tương ứng trong mảng `results`. - Giảm `remaining`; khi `remaining === 0` => resolve chính nó với `results`. 3. Nếu *bất kỳ* lúc nào* một promise con `\_reject\_` - Promise chung `\_reject\_` tức thì cùng *lý do* đó. - Các promise con khác vẫn tiếp tục chạy nhưng kết quả của chúng bị **bỏ qua**. 4. **Edge cases** - Truyền array rỗng => resolve ngay với `[]`. - Nếu promise con self-fulfilled bằng chính `allPromise` => TypeError (circular). #### 3. Tự viết mini-implementation (ES6) ``js function myAll(iterable) { return new Promise((resolve, reject) => { const values = Array.from(iterable); const len = values.length; if (len === 0) return resolve([]); let remaining = len; const results = new Array(len); values.forEach((val, i) => { Promise.resolve(val).then(res => { results[i] = res; if (--remaining === 0) resolve(results); }, err => reject(err) // first error wins); }); }); `` Điểm chính: - Bao bọc mỗi phần tử bằng `Promise.resolve` để hỗ trợ value sync. - “First-error-wins” → reject sớm, không chờ phần còn lại. #### 4. Mô hình lỗi & chiến lược thực tế | Kịch bản | Hành vi mặc định | Chiến lược gợi ý | |-----|-----|-----| | Một promise lỗi | `Promise.all` reject ngay; chuỗi `.then` kế tiếp *không* chạy | - Bọc `.catch` cục bộ cho từng promise nếu muốn bỏ qua lỗi. | | Muốn “tiếp tục” dù vài cái hỏng | Dùng `Promise.allSettled` (ES2020) trả về trạng thái & value/error của *tất cả* | - | Cần hủy request khác khi 1 cái fail | Gắn `AbortController` hay logic cancel riêng bên trong callback reject | - | Muốn lấy *tất cả lỗi* | Tự gom bằng `allSettled` rồi lọc những item `{status:'rejected'}` | - | ##### Bọc lỗi cục bộ ``js const safe = p => p.catch(err => ({error: err})); Promise.all([safe(p1), safe(p2)]).then(res => { // res[i].error có thể tồn tại }); `` #### 5. Ví dụ “dummy” đầy đủ ``js function sleep(ms, v) { return new Promise(r => setTimeout(() => r(v), ms)); } function fail(ms, reason) { return new Promise((_,

rj) => setTimeout(() => rj(reason), ms)); } Promise.all([sleep(100, 'A'), sleep(50, 'B'), fail(80, new Error('Boom')),],).then(console.log) .catch(e => console.error('✖', e.message)); /* 50 ms : B resolved (hold) 80 ms : Boom → all reject ngay 100 ms: A finish nhưng bị bỏ qua Console: ✖ Boom */ ``
 ### 6. Mở rộng: so sánh các "đội" Promise tập thể | API | Resolve khi | Reject khi | Kết quả | Use-case | |-----|-----|-----|-----|-----| | `Promise.all` | Tất cả fulfilled | Bất kỳ reject | `[values...]` | Load đồng loạt & cần *tất cả* dữ liệu | | `Promise.race` | Một settle đầu tiên (fulfill/reject) | - | value/err đầu | Chi timeout, fetch first-arrived | | `Promise.any` | Một fulfilled đầu tiên | Tất cả reject | value hoặc `AggregateError` | Lấy **một** kết quả ok, bỏ qua thất bại khác | | `Promise.allSettled` | Tất cả settle | - |
 `[{status,value|reason}]` | Muốn thống kê kết thúc & lỗi/ok song hành | ### 7. Ghi nhớ nhanh cho phỏng vấn - **Thứ tự output** = thứ tự input, không phụ thuộc thời gian resolve. - Lỗi đầu tiên chặn chuỗi `.then`, hãy `.catch` ngay sau `all` khi cần. - Với array rỗng → resolve ngay (Edge-case phổ biến bị quên). - Thực thi song song thật sự phụ thuộc async bên trong (I/O, worker, setTimeout), chứ bản thân `all` không tạo luồng. > **Chốt:** `Promise.all` là "bộ kết khoá" giúp chạy nhiều công việc bất đồng bộ song song, đơn giản hóa flow control; nắm nội tạng counter + resolve sớm khi gặp lỗi là chìa khoá để tối ưu và xử lý sự cố đúng cách. --- ## Đối xứng số (palindrome number) là gì? - **Định nghĩa:** Một số nguyên dương được gọi là *đối xứng* khi dãy chữ số của nó đọc xuôi hay ngược đều giống nhau. Ví dụ: 7, 33, 121, 1331. - **Thuộc tính:**
 1. Chữ số đầu và cuối luôn giống nhau. 2. Nếu độ dài lẻ, chữ số ở giữa có thể là bất cứ giá trị nào (0-9). 3. Nếu độ dài chẵn, mọi cặp vị trí đối xứng trong chuỗi đều bằng nhau. ## Bài toán: Liệt kê tất cả số đối xứng trong đoạn Giới hạn 10,000 ⇒ ta chỉ cần xét tối đa 5 chữ số (10,000 gồm 5 chữ số "10000", nhưng bản thân 10,000 không đối xứng vì 1≠0). ### Cách tiếp cận 1 – Duyệt thẳng + kiểm tra đảo chuỗi (dễ hiểu) ``js function isPalindrome(n) { const s = String(n); return s === s.split('').reverse().join(''); } function palindromesTo10000() { const res = []; for (let i = 1; i < 10,000 ⇒ bỏ. ``js function generatePalindromes() { const res = []; // 1 digit for (let d = 1; d < 10; d++) { res.push(d); } // 2 digit for (let d1 = 1; d1 < 10; d1++) { for (let d2 = 0; d2 < 10; d2++) { res.push(d1*10+d2); } } // 3 digit for (let d1 = 1; d1 < 10; d1++) { for (let d2 = 0; d2 < 10; d2++) { for (let d3 = 0; d3 < 10; d3++) { res.push(d1*100+d2*10+d3); } } } // 4 digit for (let d1 = 1; d1 < 10; d1++) { for (let d2 = 0; d2 < 10; d2++) { for (let d3 = 0; d3 < 10; d3++) { for (let d4 = 0; d4 < 10; d4++) { res.push(d1*1000+d2*100+d3*10+d4); } } } } // 5 digit for (let d1 = 1; d1 < 10; d1++) { for (let d2 = 0; d2 < 10; d2++) { for (let d3 = 0; d3 < 10; d3++) { for (let d4 = 0; d4 < 10; d4++) { for (let d5 = 0; d5 < 10; d5++) { res.push(d1*10000+d2*1000+d3*100+d4*10+d5); } } } } } } return res; } ``
 Nhớ: Trong phỏng vấn, cách 1 đủ pass, nhưng nên thêm cách 2 cho thấy bạn hiểu tính chất toán & tối ưu hoá. --- # Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt): 第 82 题：周一算法题之「移动零」 给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。 示例: ini 代码解读 复制代码 输入: [0,1,0,3,12] 输出: [1,3,12,0,0] 说明: 必须在原数组上操作，不能拷贝额外的数组。 尽量减少操作次数。 ♦ BÀI 82 – "MOVE ZEROES" Đề: Cho mảng `nums`, di chuyển **tất cả các phần tử bằng 0** về cuối mảng, **giữ nguyên thứ tự** của các phần tử khác, **không** dùng mảng phụ, **tối thiểu thao tác** ghi. ----- I. TƯ DUY GỐC 0 đóng vai trò "rác" phải gom về cuối, còn các số ≠ 0 phải giữ thứ tự. → Vấn đề trở thành **"lọc" non-zero sang đầu, sau đó điền 0 ở phần đuôi. Nếu ta có con trỏ `write` chỉ vị trí kế tiếp cần gán non-zero, mỗi khi gặp giá trị non-zero ở chỉ số `read`: 1. Ghi `nums[write] = nums[read]` (nếu `read ≠ write`). 2. Tăng `write++`. Sau vòng lặp, mọi non-zero đã ổn định ở đầu (đúng trật tự). Cuối cùng set toàn bộ đoạn `[write ... n-1]` thành 0. - Time O(n) – quét một lần. - Space O(1) – chỉ vài biến. - Số lần **ghi** tối đa n (non-zero) + n (set 0) nhưng có thể giảm (xem § III.2). ----- II. IMPLEMENTATION (JS) ``js function moveZeroes(nums) { let write = 0; // vị trí ghi kế tiếp // 1) Gom non-zero lên đầu (giữ thứ tự) for (let read = 0; read < nums.length; read++) { if (nums[read] !== 0) { nums[write++] = nums[read]; // copy hoặc tự-ghi } } // 2) Lấp 0 cho phần còn lại while (write < nums.length) { nums[write++] = 0; } /* Demo */ const nums = [0, 1, 0, 3, 12]; moveZeroes(nums); console.log(nums); // [1, 3, 12, 0, 0] `` -----

----- III. PHÂN TÍCH SÂU & TỐI ƯU 1. Ổn định (stable) – Thứ tự non-zero được copy tuần tự, nên luôn ổn định. 2. Tối thiểu số ghi - Nếu `read === write` (tức phần tử vốn đã ở đúng chỗ) ta **không cần** overwrite. - Ghi 2 pha tách biệt (non-zero + 0) tốt cho cache vì hai đoạn liên tiếp. 3. Mô hình "two-pointer" vs "swap" a. **Copy** (ở trên) – ổn định, ít swap. b. **Swap tại chỗ** – khi gặp non-zero, swap với `nums[lastZero]` rồi tăng cả hai con trỏ; không cần pha lấp 0: ``js
function moveZeroesSwap(nums) { let lastZero = 0; for (let i = 0; i < nums.length; i++) { if
(nums[i] !== 0) { [nums[lastZero], nums[i]] = [nums[i], nums[lastZero]]; lastZero++; } } `` –
Cũng O(n) / O(1) nhưng có thể nhiều swap hơn; vẫn ổn định vì vị trí `i` luôn $\geq$ `lastZero`. 4. Khung
so sánh | Phương án | Ghi non-zero | Ghi 0 | Swap | Ổn định | |-----|-----|-----|---
---|-----| | Copy+fill | $\leq$ (#non-zero) | (#zero) | 0 | ✓ | | Swap | (#non-zero) $\times \leq 2$ | 0 | $\leq$ (#non-
zero) | ✓ | 5. Edge-cases – Toàn 0 → vòng copy không ghi, pha 2 ghi n số 0 (nhưng thực chất
đã 0, có thể bỏ check). – Không 0 → write=n, pha 2 bỏ qua, 0 ghi. -----

----- IV. WALK-THROUGH DUMMY Input `` `` read write nums 0 0
[0,1,0,3,12] // 0 ⇒ skip 1 0 [1,1,0,3,12] // copy 1, write=1 2 1 [1,1,0,3,12] // 0 ⇒ skip 3 1 [1,3,0,3,12]
// copy 3, write=2 4 2 [1,3,12,3,12] // copy 12, write=3 -- fill -- 3 - [1,3,12,0,12] // set 0 4
[1,3,12,0,0] `` ----- V. MỞ RỘNG &

BIẾN THỂ 1. **Đưa 0 về đầu, giữ thứ tự** – Đảo chiều con trỏ hoặc đảo mảng rồi dùng thuật
toán cũ. 2. **Đếm #swap tối thiểu** – Dùng công thức: số zero trước mỗi non-zero = tổng swap
cần (ứng dụng measure performance). 3. **Stream / Linked List** – Với list đơn: giữ hai con trỏ
prevZero & cur để unlink/append node 0 về cuối – O(n), O(1). 4. **SIMD / TypedArray lớn** –
Quét trước positions của non-zero vào bitmask, rồi `packed move` (AVX512 `vpmov`), thực tế
đẩy throughput $\times 4$ so với vòng for, áp dụng audio buffer. 5. **Ngôn ngữ khác** – C++ 20:
``cpp void moveZeroes(std::vector& v){ auto it = std::remove(v.begin(), v.end(), 0); // stable
remove-erase idiom std::fill(it, v.end(), 0); } `` – `std::remove` gom non-zero về đầu, trả iterator
đuôi. ----- VI. CÂU HỎI PHÒNG VẤN

LIÊN QUAN 1. Làm thế nào giảm **cache miss**? → gom phase copy & fill thành 2 pass liên
tiếp, không xen kẽ. 2. Nếu yêu cầu **chỉ tối đa k phép hoán vị**? → cần tính phân bố zero để
quyết định swap hay copy. 3. Tại sao không dùng `filter` + `concat`? → tạo mảng mới → vi
phạm đề bài. 4. Khác biệt với bài "Sort Colors (Dutch Flag)" ? → Sort Colors cần đổi thứ tự, bài
này phải **giữ** relative order. -----

KẾT LUẬN Thuật toán hai con trỏ (copy-&-fill) là lựa chọn tối ưu: O(n) thời gian, O(1) bộ nhớ, số
ghi tối thiểu, dễ chứng minh tính ổn định. Hiểu sâu cơ chế giúp bạn mở rộng sang stream, SIMD,
linked-list hoặc tối ưu cache trong những hệ thống xử lý dữ liệu lớn. --- ## var, let, const – ba
"kiểu khai báo" của JavaScript nhìn dưới kính hiển vi V8 ### 1. Tóm tắt hành vi bề mặt | Thuộc
tính | `var` | `let` | `const` | |-----|-----|-----|-----| | Phạm vi (scope) | Function-
scope | Block-scope | Block-scope | | Hoisting | Có – gán `undefined` | Có – nhưng **TDZ** |
Có – TDZ | | Gán lại (re-assign) | ✓ | ✓ | ✗ | | Khai báo lại (re-declare trong cùng scope) | ✓ | ✗ |
✗ | | Thuộc tính `window` trong script top-level | ✓ | ✗ | ✗ | *TDZ: Temporal Dead Zone – vùng
"chết tạm thời" từ đầu block tới dòng khai báo; truy cập biến lúc này ném ReferenceError. ### 2.
Chu kỳ đời biến bên trong Runtime 1. **Parsing phase (Creation)** ECMA-262 yêu cầu engine
lập **Environment Record** (ER) cho mỗi scope. - `var` đặt tại **Variable Environment** của
Function/Global ER. - `let`/`const` tạo mục trong **Lexical Environment** kèm flag
`uninitialized`. 2. **Hoisting & TDZ** - `var` mục tiêu được **Initialize = undefined** ngay khi
ER tạo ✓, nên đọc trước khai báo chỉ nhận `undefined`. - `let`/`const` giữ trạng thái
"uninitialized" → bất kỳ GetBindingValue trước khi Initialize sinh **ReferenceError** (đó là TDZ).
- Initialization của `let`/`const` diễn ra tại dòng khai báo; `const` thêm flag `immutable`. 3.

****Execution phase**** - Khi interpreter gặp `let x = 5;` nó gán giá trị & chuyển binding thành ***initialized, mutable***. - Với `const`, sau gán đầu tiên flag ***mutable*** đặt `false`; các `PutValue` sau → ****TypeError: Assignment to constant variable****. 4. ****Garbage collection**** Biến block-scope được GC khi ****Lexical Environment**** rời phạm vi. `var` sống đến hết Activation Record (toàn bộ function) – nguy cơ rò RAM nếu closure không cần. ### 3. Thực thi sâu trong V8 (Chrome / Node)

- **One-Bytecode pass**** - Parser chuyển mã thành ****Ignition bytecode****. - Bytecode chứa opcodes `CreateMutableBinding`, `CreateImmutableBinding`, `SetName`, `GetName`... 2. ****`let`/`const` & TDZ check**** - Opcode `LdaLookupSlot` khi nhìn thấy binding ***uninitialized*** → builtin `ThrowReferenceErrorIfHole`. 3. ****优化**** (Inline Caching & TurboFan) - Vì `let`/`const`` block-scope và không xuất hiện trong prototype chain, TurboFan có thể **\*\*skip dynamic lookup\*\*** sau lần đầu (`ContextSlot`` fast path). - `var`` reference top-level → property LoadIC của global object, chậm & ít inline được. 4. **\*\*Const violation\*\*** - `StoreInArrayLiteral`, `SetNameStrict`... gọi `Runtime_ThrowConstAssignIfStrict` khi flag `immutable = true`. ### 4. Khai báo lồng & Shadowing ``js let x = 1; { const x = 2; // shadow bằng bản binding mới trong block var y = x; // y function-scope, trả giá trị 2 } console.log(x); // 1 console.log(y); // 2 `` - Mỗi level block tạo ****context chain****: truy vấn tên duyệt từ trong ra ngoài. - Shadow không ảnh hưởng binding cha (đối lập hoàn toàn với biến capture của closure). ### 5. Tối ưu hoá & Best-practice
- **Giảm hoisting bug**** Hãy ưu tiên `let`/`const`` để engine chặn sớm thuộc tính undefined. 2. **\*\*Đóng băng trạng thái bằng `const`\*\*** - Bất biến tham chiếu – **\*không\*** bất biến giá trị: ``js const arr = []; arr.push(1); // OK arr = [2]; // TypeError `` - Freeze deep: `Object.freeze`. 3. ****TDZ lợi cho tuần tự hoá module**** Import ES6 module cũng nằm trong TDZ: circular import được phát hiện ngay. 4. ****Memory locality**** Block-scope biến giải phóng sớm → heap pressure giảm; trong V8, context chết được tách khỏi closure list → sweep nhanh hơn. ### 6. Ví dụ “dummy” mô phỏng lỗi phổ biến ``js function foo(flag) { if (flag) { var v = 'var'; // hoisted let l = 'let'; } console.log(v); // 'var' (undefined nếu flag=false) console.log(l); // ReferenceError (TDZ) } `` Giúp junior hiểu vì sao `let`/`const`` an toàn hơn. ### Kết luận nhanh - **\*\*var\*\***: di sản ES3, function-scope, hoisting to undefined, dễ sinh bug. - **\*\*let\*\***: block-scope, TDZ, cho phép gán lại → lựa chọn mặc định. - **\*\*const\*\***: như `let`` nhưng thêm bất biến tham chiếu, compiler tối ưu hơn, dùng cho hằng & giá trị không tái gán. Hiểu cơ chế Environment Record, TDZ và flag ***immutable*** giúp thiết kế code rõ ràng, tránh hoisting-bug và tận dụng tối đa pipeline tối ưu của V8. --- ## Yêu cầu Tạo hàm `add`` sao cho nó: `` add(1) → 1 add(1)(2) → 3 add(1)(2)(3) → 6 add(1)(2, 3) → 6 add(1, 2)(3) → 6 add(1, 2, 3) → 6 `` Nói cách khác, `add`` phải: 1. Nhận ****bất kỳ số lượng tham số**** ở mỗi lần gọi. 2. Có thể ****chuỗi nhiều lần**** (currying không giới hạn). 3. Khi chuyển đổi sang số (in ra console, so sánh, tính toán) phải cho kết quả là ****tổng tất cả tham số đã truyền từ đầu tới cuối chuỗi****. ## Cơ chế cốt lõi JavaScript cho phép một hàm ****là đối tượng****; ta có thể gán thêm phương thức hoặc ghi đè phép chuyển đổi kiểu của nó. Ý tưởng: 1. Giữ biến `sum`` tích lũy. 2. Mỗi lần gọi lại (`add(...)(...)`) trả về **\*\*cùng một hàm\*\*** có closure giữ `sum``. 3. Ghi đè ****toString**** hoặc ****valueOf**** (ES3) – hoặc trong ES6+, dùng ****Symbol.toPrimitive**** – để khi hàm bị ép sang primitive, trả về `sum``. ## Triển khai chuẩn ES6+ ``js function add(...args) { // 1. khởi tạo tổng ban đầu let sum = args.reduce((a, b) => a + b, 0); // 2. định nghĩa hàm tạm nhận thêm tham số function adder(...more) { sum += more.reduce((a, b) => a + b, 0); return adder; // trả chính nó để chain tiếp } // 3. custom conversion: number, string, +, console.log adder[Symbol.toPrimitive] = () => sum; // ưu tiên ES6 adder.toString = () => String(sum); // dự phòng adder.valueOf = () => sum; // dự phòng return adder; } `` ### Thử nghiệm ``js console.log(add(1)); // 1 console.log(add(1)(2)); // 3 console.log(add(1)(2)(3)); // 6 console.log(add(1)(2, 3)); // 6 console.log(add(1, 2)(3)); // 6

`console.log( add(1, 2, 3) ); // 6` // vẫn là hàm nên có thể tiếp tục: `const sum10 = add(1)(2)(3)(4); console.log(+sum10); // 10` (unary + ép primitive)

Giải thích chi tiết 1. **Closure giữ state**
 – Biến `sum` sống trong phạm vi lexicographic của `adder`, vì vậy mỗi lần `adder` tái sử dụng chính nó, `sum` vẫn còn.

2. **Currying linh hoạt** – Hàm `adder` luôn *return itself* ⇒ chuỗi gọi bao lâu tùy ý.

3. **Ép kiểu (type coercion)** – Khi bạn `console.log(adder)` hay `Number(adder)`: * JS cố gọi `Symbol.toPrimitive` (nếu có) với hint `'number'` | `'string'` | `'default'`. * Nếu không có, rơi xuống `valueOf`, rồi `toString`. – Trả về `sum` ⇒ hàm **hành xử như số**.

4. **Không giới hạn độ dài chuỗi** – Mỗi call chỉ cập nhật `sum` rồi return `adder`; dừng lúc nào cũng được, kết quả khi coercion.

Biến thể & nâng cao 1. **Tạo `add.end()`**
 Nếu muốn ép người dùng gọi `end()` thay vì rely on coercion: `js adder.end = () ⇒ sum;`

2. **Hỗ trợ `BigInt`** Phát hiện kiểu tham số, duy trì hai biến `sumNumber` & `sumBigInt`.

3. **Đảm bảo “thuần”** Hàm hiện sửa đổi đối tượng hàm (monkey-patch). Nếu cần bất biến, hãy `Object.freeze(adder)`.

4. **Generic reducer** Tổng quát hoá thành `createCombiner(fn)` để làm `multiply`, `concat`, v.v.

Tổng kết Trick nằm ở chỗ: – **Currying + closure để tích lũy giá trị. – **Override coercion** (`Symbol.toPrimitive`/`valueOf`) để biến *hàm* thành *số* khi cần. Nhờ đó `add` vừa có thể gọi lặp `add(...)(...)` vô hạn, vừa cho ra kết quả đúng khi in hoặc tham gia phép toán.**

--- ### 1. (react-router) vs. thẻ `<Tiêu chí>` (HTML thuần) | (react-router-dom) | Cơ chế điều hướng | Trình duyệt **tải lại toàn bộ trang (full-page reload). | **Đẩy URL vào History API** → React-Router đổi component, **không reload**. | Hiệu năng | Mất round-trip HTTP, xoá sạch trạng thái JS, CSS, store. | SPA chuyển trang tức thì, giữ nguyên state, cache. | SEO & hành vi gốc | Hoạt động kể cả khi JS tắt; hỗ trợ “Right-click → Open in new tab”. | Phụ thuộc JavaScript; vẫn hỗ trợ Ctrl/Cmd-click, middle-click nhờ prop `target` & logic mặc định. | Thuộc tính `href`, `target`, `download`, ... | `to`, `replace`, `state`, `reloadDocument`, `prefetch`, v.v. | Prefetch dữ liệu | Không | Có thể hook `useLoaderData` (v6.10+) hoặc `prefetch="intent"`. | Phân quyền / điều kiện | Phải tự viết JS ngăn click. | Có thể wrap `</>` để redirect có điều kiện. | Điểm mấu chốt: **không** để trình duyệt gửi request mới → UX mượt, state không mất; React-Router chỉ thay đổi **client-side route**.**

2. Vì sao KHÔNG dùng thẻ `<thuần>` trong SPA?

1. Full reload xoá Redux/Context, scroll, WebSocket, v.v.
2. Layout “flash” (trắng màn hình) cho tới khi bundle tải lại.
3. Không tận dụng được code-splitting & caching của router. Do đó trong phạm vi SPA, ta **dùng `<cho navigation bên trong ứng dụng>`; `<chỉ dành cho: - Tải file, anchor ngoài trang. - Liên kết external (href="https://..."). - Kịch bản cần tải lại thật sự (ex: logout → GET /login).`**

3. Cách “vô hiệu hoá” hành vi mặc định (prevent default) của `<và điều hướng bằng code>`

3.1. Classic DOM

```

jsx function MyLink() { const handleClick = e ⇒ { e.preventDefault(); // chặn reload window.history.pushState({}, "", '/dashboard'); // thay URL // sau đó tùy SPA: phát sự kiện hoặc react-router lắng nghe popstate }; return Go; }
  
```

 Nhược: tự quản popstate, scroll, accessibility. React-Router đã làm sẵn.

3.2. Với `react-router-dom` v6 (khuyến nghị) Sử dụng hook `useNavigate`:


```

jsx import { useNavigate } from 'react-router-dom'; function MyLink() { const navigate = useNavigate(); const handleClick = e ⇒ { e.preventDefault(); // ngăn reload navigate('/dashboard', { replace: false, state: { from: 'home' } }); }; return Go; }
  
```

 – `navigate(path)` thực hiện cùng cơ chế mà `<dùng bên trong>`. – Vẫn cho phép bạn kiểm soát **điều kiện**, **log**, **analytics** trước khi chuyển.

3.3. Biến thể “button trông như link”

```

jsx import { Link } from 'react-router-dom'; Go
  
```

 Không cần `<React-Router render>` dưới hood và xử lý `onClick` + `preventDefault()` tự động.

4. Một số lưu ý nâng cao

1. – nếu thật sự muốn reload full trang vẫn có thể.
2. Prefetch: `<tải dữ liệu trước khi user click (RR v6.10+)>`.
3. Accessibility: Khi tự viết `<nhớ thuộc tính role="link">` & `onKeyDown`

(Enter/Space) cho keyboard users. 4. SEO / SSR (Next.js, Remix): `` vẫn được HTML hoá ở server; crawler thấy ``. ### 5. Cheatsheet chọn thành phần | Tình huống | Thành phần nên dùng | |-----|-----| | Chuyển trang nội bộ SPA | `` | | Redirect có điều kiện (login guard) | `useNavigate` hoặc `` | | Submit form rồi chuyển | `useNavigate` trong `onSubmit` | | Liên kết external / download file | `` thuần | | Cần reload để reset app state | `` hoặc `` | \*\*Kết luận\*\* - ``: cơ chế truyền thống, gửi request mới. - ``: wrapper của `react-router`, gọi `e.preventDefault()` sau đó `history.pushState()` → navigation tức thời, giữ nguyên SPA state. - Nếu bắt buộc dùng `` nhưng vẫn muốn SPA navigation, bạn chỉ cần `e.preventDefault()` và gọi `navigate()` (hoặc `history.pushState`) bằng JavaScript. --- ## BÀI 86 – "TWO SUM" – Bài toán kinh điển trong Data Structures & Algorithms ### I. PHÂN TÍCH BÀI TOÁN **Đầu vào:** Mảng `nums` và giá trị đích `target` **Đầu ra:** Chỉ số của hai phần tử có tổng bằng `target` **Ràng buộc:** - Mỗi input có duy nhất một nghiệm - Không được dùng cùng phần tử hai lần - Trả về chỉ số, không phải giá trị #### II. BA CÁCH TIẾP CẬN CHÍNH #### 1. Brute Force – $O(n^2)$ approach ``js function twoSumBruteForce(nums, target) { const n = nums.length; for (let i = 0; i < n - 1; i++) { for (let j = i + 1; j < n; j++) { if (nums[i] + nums[j] === target) { return [i, j]; } } } return []; // không tìm thấy } `` **Đặc điểm:** - Time: $O(n^2)$ – hai vòng lặp lồng nhau - Space: $O(1)$ – chỉ dùng biến tạm - **Ưu:** Đơn giản, không cần thêm bộ nhớ - **Nhược:** Chậm với mảng lớn #### 2. Hash Table – $O(n)$ optimal solution ``js function twoSum(nums, target) { const map = new Map(); // value → index for (let i = 0; i < nums.length; i++) { const complement = target - nums[i]; if (map.has(complement)) { return [map.get(complement), i]; } map.set(nums[i], i); } return []; // không tìm thấy } `` **Cách thức hoạt động:** 1. Duyệt qua từng phần tử `nums[i]` 2. Tính `complement = target - nums[i]` (số cần tìm) 3. Kiểm tra xem `complement` đã xuất hiện trong hash table chưa 4. Nếu có → trả về `[index của complement, i]` 5. Nếu không → lưu `nums[i] : i` vào hash table **Đặc điểm:** - Time: $O(n)$ – một lượt qua mảng - Space: $O(n)$ – hash table lưu tối đa n phần tử - **Tối ưu nhất** cho trường hợp tổng quát #### 3. Two Pointers – $O(n \log n)$ (nếu được phép sắp xếp) ``js function twoSumSorted(nums, target) { // Tạo mảng [value, originalIndex] và sắp xếp theo value const indexed = nums.map((val, idx) => [val, idx]).sort((a, b) => a[0] - b[0]); let left = 0, right = indexed.length - 1; while (left < right) { const sum = indexed[left][0] + indexed[right][0]; if (sum === target) { return [indexed[left][1], indexed[right][1]].sort((a, b) => a - b); } else if (sum < target) { left++; } else { right--; } } return []; } `` \*\*Đặc điểm:\*\* - Time:  $O(n \log n)$  – do sort, two pointers chỉ là  $O(n)$  - Space:  $O(n)$  – mảng indexed - \*\*Ưu:\*\* Intuitive, dễ mở rộng cho "3Sum", "4Sum" - \*\*Nhược:\*\* Chậm hơn hash table, cần giữ original indices ### III. DEMO VÀ WALK-THROUGH ``js // Input: nums = [2, 7, 11, 15], target = 9 /* Hash Table approach step-by-step: i=0: nums[0]=2, complement=7, map={} → map.set(2,0) i=1: nums[1]=7, complement=2, map={2:0} → tìm thấy! return [0,1] */ console.log(twoSum([2, 7, 11, 15], 9)); // [0, 1] console.log(twoSum([3, 2, 4], 6)); // [1, 2] console.log(twoSum([3, 3], 6)); // [0, 1] `` ### IV. BIẾN THỂ VÀ MỞ RỘNG NÂNG CAO #### 1. Two Sum II – Input Array Is Sorted Nếu mảng đã được sắp xếp, two pointers là tối ưu: ``js function twoSumII(numbers, target) { let left = 0, right = numbers.length - 1; while (left < right) { const sum = numbers[left] + numbers[right]; if (sum === target) return [left + 1, right + 1]; // 1-indexed else if (sum < target) left++; else right--; } } `` #### 2. Two Sum – All Pairs Trả về tất cả cặp có tổng bằng target: ``js function twoSumAllPairs(nums, target) { const map = new Map(); const result = []; for (let i = 0; i < nums.length; i++) { const complement = target - nums[i]; if (map.has(complement)) { // Lấy tất cả indices của complement for (const j of map.get(complement)) { result.push([j, i]); } } // Lưu index hiện tại if (!map.has(nums[i])) map.set(nums[i], []); map.get(nums[i]).push(i); } return result; } `` #### 3. Two Sum – Closest Pair Tìm cặp có tổng gần target nhất: ``js function

```

twoSumClosest(nums, target) { nums.sort((a, b) => a - b); let left = 0, right = nums.length - 1; let
closest = Infinity; let result = []; while (left < right) { const sum = nums[left] + nums[right]; const
diff = Math.abs(sum - target); if (diff < closest) { closest = diff; result = [nums[left], nums[right]];
} if (sum < target) left++; else right--; } return result; }
```
4. Two Sum – With Duplicates Handling ```js
function twoSumWithDuplicates(nums, target) { const seen = new Set(); const
used = new Set(); const result = []; for (let i = 0; i < nums.length; i++) { const complement =
target - nums[i]; if (seen.has(complement) && !used.has(nums[i]) && !used.has(complement)) {
result.push([nums[i], complement]); used.add(nums[i]); used.add(complement); }
seen.add(nums[i]); } return result; }
```
#### V. TỐI ƯU HÓA VÀ PERFORMANCE TUNING #### 1.
Memory-efficient Hash Table ```js
function twoSumOptimized(nums, target) { const map =
Object.create(null); // không có prototype chain for (let i = 0; i < nums.length; i++) { const
complement = target - nums[i]; if (complement in map) { return [map[complement], i]; }
map[nums[i]] = i; } return []; }
```
2. Early Termination Strategies ```js
function
twoSumEarlyExit(nums, target) { // Nếu target quá lớn so với max có thể, exit sớm const max =
Math.max(...nums); if (target > 2 * max) return []; const map = new Map(); for (let i = 0; i <
nums.length; i++) { const complement = target - nums[i]; // Early exit nếu complement quá lớn if
(complement > max) continue; if (map.has(complement)) { return [map.get(complement), i]; }
map.set(nums[i], i); } return []; }
```
#### VI. ỨNG DỤNG THỰC TẾ #### 1. System Design –
Cache Optimization ```js
class TwoSumCache { constructor() { this.cache = new Map(); //
(nums_hash, target) -> result } twoSum(nums, target) { const key = this.hashArray(nums) + '_' +
target; if (this.cache.has(key)) { return this.cache.get(key); } const result =
this.computeTwoSum(nums, target); this.cache.set(key, result); return result; } hashArray(nums)
{ return nums.sort().join(','); } computeTwoSum(nums, target) { // Standard two sum
implementation } }
```
2. Database Query Optimization ```sql
-- Tương đương SQL cho Two
Sum SELECT a.value, b.value, a.index, b.index FROM numbers a JOIN numbers b ON a.value +
b.value = @target WHERE a.index < b.index;
```
#### 3. Distributed Computing ```js
//
MapReduce pattern cho Two Sum trên big data function mapTwoSum(partition, target) { const
results = []; const map = new Map(); for (const [value, globalIndex] of partition) { const
complement = target - value; if (map.has(complement)) { results.push([map.get(complement),
globalIndex]); } map.set(value, globalIndex); } return results; }
```
VII. COMPLEXITY
ANALYSIS SÂU | Approach | Time | Space | Stability | Scalability | -----|-----|-----|-----
-----|-----| Brute Force | O(n²) | O(1) | Stable | Poor | Hash Table | O(n) | O(n) | Stable
| Excellent | Two Pointers | O(n log n) | O(n) | Stable | Good | **Cache Performance:** - Hash
table có **locality tốt** hơn brute force - Two pointers có **memory access pattern**
predictable
VIII. KẾT LUẬN VÀ TAKEAWAYS 1. **Hash table approach** là solution chuẩn
cho Two Sum 2. **Space-time tradeoff** classic: O(1) space vs O(n) time 3. **Pattern này**
mở rộng thành 3Sum, 4Sum, k-Sum 4. **Real-world applications:** recommendation systems,
financial pair trading, data deduplication **Câu hỏi follow-up thường gặp:** - Nếu mảng đã
sorted thì sao? - Nếu có nhiều solutions? - Nếu input stream (không biết trước kích thước)? -
Nếu phải tối ưu memory hơn time? Hiểu sâu Two Sum giúp nắm vững **hash-based
algorithms**, **two pointers technique**, và **search optimization** – nền tảng cho nhiều bài
toán phức tạp hơn. --- ## Cách kiểm tra "URL hợp lệ" ngay trong ô nhập liệu > Mục tiêu: Khi
người dùng gõ vào , xác định xem chuỗi đó có phải là **địa
chỉ URL hợp lệ** theo chuẩn chuẩn URL (WHATWG/ RFC 3986) và tương thích với nhu cầu sản
phẩm (http, https, ftp, mailto, v.v.).
1. Khái niệm & tiêu chuẩn cần nắm - **URL (Uniform
Resource Locator)**: chuỗi ký tự gồm các phần `scheme://userinfo@host:port/path?`

```

`query#fragment``. - **WHATWG URL Standard** (trình duyệt hiện đại áp dụng) bổ sung quy tắc xử lý lenient (tự động encode, punycode domain...). - **RFC 3986** mô tả cú pháp chính thống, nghiêm ngặt hơn. ⇒ Trong thực tế front-end, ta **tận dụng bộ parser URL có sẵn** của trình duyệt thay vì tự viết regex phức tạp. **### 2. Các phương pháp kiểm tra** | Phương pháp | Ưu điểm | Nhược điểm | Thích hợp | |-----|-----|-----|-----| | `Regex` tự viết | Không cần API | Rất khó đúng 100%, khó bảo trì | Thao tác nhanh, pattern hẹp | | Dùng `HTMLInputElement.checkValidity()` với `type="url"` | Native, tự động ESLint, hỗ trợ pattern | Bị giới hạn bởi HTML spec, bỏ qua scheme custom | Form cơ bản | | Dùng `new URL()` (WHATWG parser) | Độ chính xác cao, hỗ trợ international domain, dễ bắt lỗi | Throw exception → cần `try/catch`, chấp nhận mọi scheme | SPA, logic JS thuần | | Thư viện bên ngoài (validator.js, yup) | Nhiều tùy chọn, có whitelist/blacklist | Kích thước bundle, phụ thuộc | Dự án lớn, nhiều rule | **### 3. Giải pháp khuyến nghị (modern browser)** `html`

`html` Giải thích: 1. `new URL(str)` tự động: - Chuẩn hoá punycode (`https://điện.toán.vn`` OK). - Thêm mặc định `"about:blank"` base nếu chỉ cung cấp path. 2. Bất kỳ chuỗi sai cú pháp (khoảng trắng, backslash, v.v.) sẽ phát sinh `TypeError` → bắt bằng `try/catch`. 3. Bạn tùy biến rule (`protocol`, `hostname`, `port`, `pathname`, `tld`). **### 4. So sánh với `type="url"`** `html`  `html` - Trình duyệt sẽ: - Tô viền đỏ khi sai. - Ngăn submit form khi `.checkValidity() == false`. - Tuy nhiên: - **Không** báo sai lập tức nếu user gõ `example.com` (thiếu scheme). - Một số trình duyệt "khoan dung" hơn spec (Edge chấp nhận `http://` thiếu domain). ⇒ Dùng `type="url"` + `pattern` rất tiện ở HTML form truyền thống, nhưng vẫn nên kết hợp JavaScript validate nâng cao để thống nhất hành vi. **### 5. Xử lý edge-case quan trọng** 1. **Scheme tùy biến**: `mailto:`, `tel:`, `custom-app://...` - Thêm whitelist hoặc chấp nhận tất: `if (!url.protocol.endsWith(':')) return false;` 2. **IP v4/v6**: `http://127.0.0.1`, `http://[2001:db8::1]` - `URL` parser tự nhận diện. 3. **Relative URL**: `/assets/img.png` - Nếu muốn từ chối, kiểm tra `str.startsWith('http')`, vì `new URL()` với base cho phép. 4. **Internationalized domain**: `https://測試.tw` hợp lệ sau khi được encode punycode. 5. **JavaScript URL spoofing**: filter `javascript:` scheme để ngăn XSS. **### 6. Performance & UX** - Debounce validate (200–300 ms) để tránh validate liên tục khi gõ. - Hiển thị thông báo rõ ràng: "URL phải bắt đầu bằng `http://` hoặc `https://`". - Tùy màn hình mobile, có thể gợi ý bàn phím URL: `html`  `html` **### 7. Thư viện gợi ý** `bash` `npm i validator` `js` `import isURL from 'validator/lib/isURL'; isURL('https://google.com', { require_protocol: true, protocols: ['http', 'https'], allow_underscores: false, });` - `validator.js` hỗ trợ hơn 40 tùy chọn: `max_length`, `host_whitelist`, `disallow_auth`, v.v. **### 8. Kiểm thử đơn vị (Jest)** `js` `test.each([['https://google.com', true], ['ftp://example.com', false], ['javascript:alert(1)', false], ['https://điện.toán.vn', true], ['', false],])('"%s" valid = %p', (str, expected) => { expect(isValidURL(str)).toBe(expected); });` **### Kết luận** - **Đừng viết regex** bắt toàn bộ URL – rất dễ sai. - **Parser URL built-in** (`new URL()`) cho độ tin cậy cao, hỗ trợ i18n và path phức tạp. - Kết hợp với `type="url"` + `pattern` để có feedback HTML5 ngay lập tức. - Áp dụng whitelist protocol & hostname để bảo đảm an toàn (tránh `javascript:/data:`). - Luôn có unit test cho các edge-case và cập nhật khi yêu cầu business thay đổi. --- **BÀI 88 – "LIST TO TREE CONVERSION"** – Chuyển đổi danh sách phẳng thành cấu trúc cây **### I. PHÂN TÍCH BÀI TOÁN** **Đầu vào**: Danh sách phẳng với các thuộc tính `id`, `name`, `parentId` **Đầu ra**: Cấu trúc tree với thuộc tính `children` chứa các node con **Mục tiêu**: Tối ưu độ phức tạp thời gian (từ  $O(n^2)$  xuống  $O(n)$ ) **Thách thức chính**: - Thiết lập mối quan hệ parent-child hiệu quả - Xử lý thứ tự ngẫu nhiên trong input (child có thể xuất hiện trước

parent) - Tránh duplicate traversal ### II. CÁC PHƯƠNG PHÁP TIẾP CẬN ##### 1. Naive Approach –  $O(n^2)$  (KHÔNG khuyến nghị) ```js function convertNaive(list) { const result = []; // Tim tất cả root nodes (parentId = 0) const roots = list.filter(item => item.parentId === 0); function buildChildren(parent) { const children = list.filter(item => item.parentId === parent.id); if (children.length > 0) { parent.children = children.map(child => { buildChildren(child); // recursion return child; }); } return parent; } return roots.map(buildChildren); } ``` \*\*Vấn đề:\*\* Mỗi node phải scan toàn bộ list →  $O(n^2)$  time complexity ##### 2. Hash Map Approach –  $O(n)$  OPTIMAL ```js function convert(list, rootParentId = 0) { // Bước 1: Tạo hash map và khởi tạo children array const map = new Map(); const result = []; // Khởi tạo tất cả nodes trong map với children array rỗng list.forEach(item => { map.set(item.id, { ...item, children: [] }); }); // Bước 2: Xây dựng mối quan hệ parent-child list.forEach(item => { const node = map.get(item.id); if (item.parentId === rootParentId) { // Root node result.push(node); } else { // Child node - thêm vào parent's children const parent = map.get(item.parentId); if (parent) { parent.children.push(node); } } }); return result; } ``` \*\*Ưu điểm:\*\* - \*\*Time:  $O(n)$ \*\* – chỉ duyệt list 2 lần - \*\*Space:  $O(n)$ \*\* – map lưu trữ tất cả nodes - \*\*Robust:\*\* Xử lý được thứ tự ngẫu nhiên trong input ##### 3. In-place Modification (Memory-efficient) ```js function convertInPlace(list, rootParentId = 0) { const map = new Map(); const result = []; // Bước 1: Index tất cả items list.forEach(item => { item.children = []; // thêm children array trực tiếp vào object gốc map.set(item.id, item); }); // Bước 2: Xây dựng tree structure list.forEach(item => { if (item.parentId === rootParentId) { result.push(item); } else { const parent = map.get(item.parentId); if (parent) { parent.children.push(item); } } }); return result; } ``` ### III. DEMO VÀ WALK-THROUGH ```js const list = [ {id:1,name:'部A',parentId:0}, {id:2,name:'部B',parentId:0}, {id:3,name:'部C',parentId:1}, {id:4,name:'部D',parentId:1}, {id:5,name:'部E',parentId:2}, {id:6,name:'部F',parentId:3}, {id:7,name:'部G',parentId:2}, {id:8,name:'部H',parentId:4} ]; /\* Step-by-step execution: Map after step 1: { 1: {id:1, name:'部A', parentId:0, children:[]}, 2: {id:2, name:'部B', parentId:0, children:[]}, 3: {id:3, name:'部C', parentId:1, children:[]}, ... } Step 2 processing: - item.id=1, parentId=0 → add to result[] - item.id=2, parentId=0 → add to result[] - item.id=3, parentId=1 → add to map.get(1).children[] - item.id=4, parentId=1 → add to map.get(1).children[] - item.id=5, parentId=2 → add to map.get(2).children[] - item.id=6, parentId=3 → add to map.get(3).children[] - item.id=7, parentId=2 → add to map.get(2).children[] - item.id=8, parentId=4 → add to map.get(4).children[] \*/ console.log(JSON.stringify(convert(list), null, 2)); ``` ### IV. BIẾN THỂ VÀ MỞ RỘNG NÂNG CAO ##### 1. Generic Tree Builder với Custom Key Mapping ```js function convertGeneric(list, options = {}) { const { idKey = 'id', parentKey = 'parentId', childrenKey = 'children', rootValue = 0, sortFn = null } = options; const map = new Map(); const result = []; // Initialize nodes list.forEach(item => { const node = { ...item }; node[childrenKey] = []; map.set(item[idKey], node); }); // Build tree list.forEach(item => { const node = map.get(item[idKey]); const parentId = item[parentKey]; if (parentId === rootValue) { result.push(node); } else { const parent = map.get(parentId); if (parent) { parent[childrenKey].push(node); } } }); // Optional sorting if (sortFn) { const sortTree = (nodes) => { nodes.sort(sortFn); nodes.forEach(node => { if (node[childrenKey].length > 0) { sortTree(node[childrenKey]); } }); }; sortTree(result); } return result; } // Usage const tree = convertGeneric(list, { sortFn: (a, b) => a.name.localeCompare(b.name) }); ``` ##### 2. Lazy Loading Tree (Stream Processing) ```js class TreeBuilder { constructor(rootParentId = 0) { this.map = new Map(); this.result = []; this.rootParentId = rootParentId; } addNode(item) { // Add to map const node = { ...item, children: [] }; this.map.set(item.id, node); // Process existing orphaned children this.processOrphanedChildren(item.id); // Add to parent or result if

```

(item.parentId === this.rootParentId) { this.result.push(node); } else { const parent =
this.map.get(item.parentId); if (parent) { parent.children.push(node); } } }
processOrphanedChildren(parentId) { // Tìm các node đã được add trước đó nhưng chưa có
parent this.map.forEach((node, id) => { if (node.parentId === parentId && id !== parentId) {
const parent = this.map.get(parentId); if (parent && !parent.children.includes(node)) {
parent.children.push(node); } } }); } getTree() { return this.result; } } // Usage for streaming data
const builder = new TreeBuilder(); streamData.forEach(item => builder.addNode(item)); const
tree = builder.getTree(); `` #### 3. Multi-Root Forest với Path Validation ``js function
convertToForest(list, options = {}) { const { validatePath = false, maxDepth = Infinity, onCycle =
'error' // 'error' | 'ignore' | 'warn' } = options; const map = new Map(); const result = []; const
visited = new Set(); // Initialize và detect cycles list.forEach(item => { map.set(item.id, { ...item,
children: [], depth: 0 }); }); // Calculate depth và detect cycles function calculateDepth(nodeId,
path = new Set()) { if (path.has(nodeId)) { if (onCycle === 'error') { throw new Error(`Cycle
detected: ${Array.from(path)} => ${nodeId}`); } return -1; // cycle marker } const node =
map.get(nodeId); if (!node) return -1; if (node.parentId === 0) { node.depth = 0; return 0; }
path.add(nodeId); const parentDepth = calculateDepth(node.parentId, path);
path.delete(nodeId); if (parentDepth === -1) return -1; // parent has cycle node.depth =
parentDepth + 1; return node.depth; } // Calculate depths list.forEach(item => { if
(!visited.has(item.id)) { calculateDepth(item.id); visited.add(item.id); } }); // Build tree với depth
validation list.forEach(item => { const node = map.get(item.id); if (validatePath && node.depth >
maxDepth) { console.warn(`Node ${item.id} exceeds maxDepth ${maxDepth}`); return; } if
(item.parentId === 0) { result.push(node); } else { const parent = map.get(item.parentId); if
(parent && parent.depth !== -1) { // không có cycle parent.children.push(node); } } }); return
result; } `` #### 4. Performance Monitoring Wrapper ``js function convertWithMetrics(list,
rootParentId = 0) { const startTime = performance.now(); let memoryBefore = 0; if (typeof
window !== 'undefined' && window.performance?.measureUserAgentSpecificMemory) {
memoryBefore = window.performance.measureUserAgentSpecificMemory(); } // Core conversion
logic const map = new Map(); const result = []; list.forEach(item => { map.set(item.id, { ...item,
children: [] }); }); list.forEach(item => { const node = map.get(item.id); if (item.parentId ===
rootParentId) { result.push(node); } else { const parent = map.get(item.parentId); if (parent) {
parent.children.push(node); } } }); const endTime = performance.now(); // Return result với
metrics return { data: result, metrics: { executionTime: endTime - startTime, inputSize: list.length,
outputNodes: result.length, totalNodes: map.size, averageTimePerNode: (endTime - startTime) /
list.length } }; } `` #### V. ERROR HANDLING VÀ EDGE CASES ``js function convertRobust(list,
rootParentId = 0) { // Input validation if (!Array.isArray(list)) { throw new TypeError('Input must
be an array'); } if (list.length === 0) { return []; } const map = new Map(); const result = []; const
orphanNodes = []; // Validate và initialize nodes list.forEach((item, index) => { if (!item || typeof
item !== 'object') { throw new TypeError(`Invalid item at index ${index}`); } if (item.id ===
undefined || item.parentId === undefined) { throw new Error(`Missing required fields (id,
parentId) at index ${index}`); } if (map.has(item.id)) { throw new Error(`Duplicate id ${item.id}
found`); } map.set(item.id, { ...item, children: [] }); }); // Build tree và track orphans
list.forEach(item => { const node = map.get(item.id); if (item.parentId === rootParentId) {
result.push(node); } else { const parent = map.get(item.parentId); if (parent) {
parent.children.push(node); } else { orphanNodes.push(node); } } }); // Handle orphan nodes if
(orphanNodes.length > 0) { console.warn(`Found ${orphanNodes.length} orphan nodes:`);
orphanNodes.map(n => n.id); } // Option 1: Attach to result as separate roots //

```

```

result.push(...orphanNodes); // Option 2: Create synthetic parent // result.push({ // id: 'orphans',
// name: 'Orphaned Nodes', // parentId: rootParentId, // children: orphanNodes // }); // } return
result; } `` ## VI. PERFORMANCE BENCHMARK ``js function benchmarkConversions(list,
iterations = 1000) { const methods = [{ name: 'Naive O(n²)', fn: convertNaive }, { name:
'HashMap O(n)', fn: convert }, { name: 'In-place O(n)', fn: convertInPlace }]; const results = {};
methods.forEach(({ name, fn }) => { const times = []; for (let i = 0; i < iterations; i++) { const start
= performance.now(); fn([...list]); // clone để tránh mutation times.push(performance.now() -
start); } results[name] = { avg: times.reduce((a, b) => a + b) / times.length, min:
Math.min(...times), max: Math.max(...times), median: times.sort()[Math.floor(times.length / 2)] };
}); console.table(results); return results; } // Benchmark với data size khác nhau [100, 1000,
10000].forEach(size => { const bigList = generateTestData(size); console.log(`\n=== Benchmark
for ${size} items ===`); benchmarkConversions(bigList, 100); }); `` ## VII. REAL-WORLD
APPLICATIONS #### 1. Organization Chart Component (React) ``jsx function OrgChart({ data
}) { const [treeData, setTreeData] = useState([]); useEffect(() => { const tree = convert(data);
setTreeData(tree); }, [data]); const renderNode = (node, depth = 0) => (
{node.name} (ID: {node.id})

```

```

 {node.children?.map(child => renderNode(child, depth + 1))}
 </div>

```

```
);
```

```
return (
```

```
{treeData.map(node => renderNode(node))}
```

```
);
```

```
}
```

```
2. File System Explorer
```

```

``js
function convertFileSystem(fileList) {
 return convertGeneric(fileList, {
 idKey: 'path',
 parentKey: 'parentPath',
 childrenKey: 'children',
 rootValue: '/',
 sortFn: (a, b) => {
 // Folders first, then files
 if (a.type !== b.type) {
 return a.type === 'folder' ? -1 : 1;
 }
 return a.name.localeCompare(b.name);
 }
 });
}

```

### 3. Database Category Tree (SQL → JS)

```
async function loadCategoryTree(sql) {
 const rows = await sql`
 SELECT id, name, parent_id, display_order
 FROM categories
 ORDER BY parent_id, display_order
 `;

 return convertGeneric(rows, {
 parentKey: 'parent_id',
 sortFn: (a, b) => (a.display_order || 0) - (b.display_order || 0)
 });
}
```

### VIII. COMPLEXITY ANALYSIS CHI TIẾT

| Aspect            | Naive $O(n^2)$ | HashMap $O(n)$ | Memory Impact                                |
|-------------------|----------------|----------------|----------------------------------------------|
| Time              | $O(n^2)$       | $O(n)$         | HashMap wins                                 |
| Space             | $O(n)$         | $O(n)$         | Tương đương                                  |
| Cache locality    | Poor           | Good           | HashMap có sequential access pattern tốt hơn |
| Memory allocation | Fragmented     | Bulk           | HashMap allocate một lần                     |

### IX. KẾT LUẬN VÀ BEST PRACTICES

1. Luôn sử dụng **HashMap approach  $O(n)$**  cho production
2. **Validate input data** để tránh runtime errors
3. **Handle orphan nodes** một cách graceful
4. **Consider memory footprint** khi xử lý data lớn
5. **Add performance monitoring** cho critical paths
6. **Use TypeScript** để type-safe tree operations

**Pattern này** là nền tảng cho:

- Treeview components
- Nested navigation menus
- Comment threading systems
- File/folder hierarchies
- Organization charts
- Category management systems

Hiểu sâu list-to-tree conversion giúp bạn thiết kế data structures hiệu quả và tối ưu các thao tác hierarchical data trong large-scale applications.

# Promise.race() – cơ chế, nguyên lý và polyfill “tận răng”

## 1. Promise.race hoạt động ra sao?

1. **Đầu vào:** một *iterable* (thường là mảng) gồm các giá trị hoặc promise.
2. **Xử lý trước:** mỗi phần tử được “bọc” bằng `Promise.resolve(x)` để bảo đảm đều trở thành promise<sup>[1]</sup>.
3. **Luật đua:**
  - Khi *bất kỳ* promise con **settle** (resolve hoặc reject), “cuộc đua” kết thúc ngay.
  - Promise mà `race` trả về sẽ **copy** trạng thái (fulfilled/rejected) cùng value/reason của “người về đích”<sup>[1]</sup>.
4. **Edge-cases:** iterable rỗng → promise nằm mãi ở trạng thái *pending*.

Hệ quả: `race` thường dùng cho các tình huống “ai xong trước thì lấy” như timeout, truy vấn CDN kép, tải ảnh fallback ...

## 2. Tự viết Promise.race (polyfill ES6)

```
if (!Promise.racePoly) {
 Promise.racePoly = function (iterable) {
 return new Promise((resolve, reject) => {
 // Lặp qua mọi phần tử trong iterable (Array, Set, Map.values...)
 for (const p of iterable) {
 // Đảm bảo p là promise; rồi gắn chung resolve/reject
 Promise.resolve(p).then(resolve, reject);
 }
 // Nếu iterable rỗng, vòng for không chạy ⇒ promise treo
 });
 };
}
```

Giải thích nhanh:

- `Promise.resolve(p)` cho phép truyền trực tiếp giá trị đồng bộ (`'foo'`, `42...`) hoặc “thenable”.
- `then(resolve, reject)` bảo đảm: promise nào **settle trước** sẽ gọi thẳng `resolve/reject` bên ngoài.
- Mọi promise khác, dù settle sau, bị bỏ qua vì promise đã chốt trạng thái **một lần duy nhất**.

## 3. So kè với Promise.any & Promise.all

| API               | Khi <b>hoàn thành</b>    | Trạng thái trả về                                                   | Dừng sớm? | Điển hình dùng          |
|-------------------|--------------------------|---------------------------------------------------------------------|-----------|-------------------------|
| <code>race</code> | 1 promise settle         | Copy trạng thái đầu tiên                                            | ✓         | timeout, fastest mirror |
| <code>any</code>  | 1 promise <i>resolve</i> | Value đầu tiên <i>fulfilled</i> (hoặc <code>AggregateError</code> ) | ✓         | fallback dịch vụ        |

| API              | Khi <b>hoàn thành</b> | Trạng thái trả về | Dừng sớm? | Điển hình dùng        |
|------------------|-----------------------|-------------------|-----------|-----------------------|
| <code>all</code> | Tất cả <i>resolve</i> | Mảng giá trị      | <b>x</b>  | tài hàng loạt dữ liệu |

## 4. Ví dụ “dummy” – đặt timeout cho fetch

```
const fetchWithTimeout = (url, ms = 3000) => {
 const timeout = new Promise((_, reject) =>
 setTimeout(() => reject(new Error('Timeout')), ms));

 return Promise.race([fetch(url), timeout]);
};

fetchWithTimeout('/api/data', 2000)
 .then(res => res.json())
 .then(console.log)
 .catch(console.error); // nếu >2 s sẽ ném Error: Timeout
```

## 5. Lưu ý performance & pitfall

1. **Promise leak**: các promise thua cuộc vẫn tiếp tục chạy (network, timer...) → nếu tốn tài nguyên, hãy *abort* (AbortController, clearTimeout).
2. **Không “công bằng” (unfair)**: nếu nhiều promise settle **đồng thời** (cùng tick), thứ tự ai gần then trước sẽ thắng<sup>[2]</sup>.
3. **Iterable rỗng**: nhớ bắt trường hợp nhập sai (trả về pending forever).

```
function safeRace(iterable) {
 if (!iterable[Symbol.iterator]() .next().done) return Promise.racePoly(iterable);
 return Promise.reject(new Error('Empty iterable not allowed'));
}
```

## 6. Mở rộng nâng cao

- **raceSettled** (tự viết): dừng khi *n* promise đã settle, dùng đếm ngược.
- **Cancelable race**: kết hợp AbortController để huỷ promise thua.
- **“Fair race”**: xếp promise theo `performance.now()` khi chúng gửi tín hiệu rồi lấy min timestamp – phức tạp, hiếm khi cần<sup>[2]</sup>.

## 7. TL;DR nhớ 3 dòng code cốt lõi

```
Promise.racePoly = its =>
 new Promise((res, rej) => {
 for (const p of its) Promise.resolve(p).then(res, rej);
 });
```

Chỉ vậy, "trường đua" Promise.race đã hoàn thành với độ phức tạp  $O(n)$  & cực kỳ gọn.

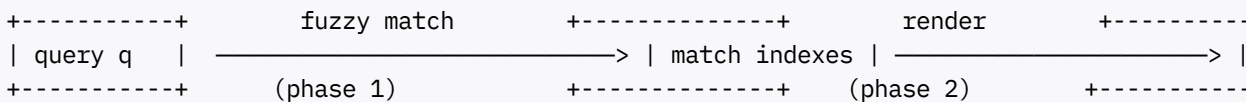


## "Keyword Highlighting" cho kết quả tìm kiếm mờ (fuzzy search)

### 1. Bài toán & bối cảnh

- **Input 1 (query):** chuỗi người dùng gõ, có thể thiếu dấu, gõ sai ký tự, không đủ từ.
- **Input 2 (text):** tiêu đề, đoạn mô tả... cần so khớp và hiển thị.
- **Yêu cầu:**
  1. Trả về *điểm khớp* (score) & vị trí matching theo thuật toán fuzzy.
  2. Bao-hàm **bôi đậm** (highlight) *chính xác* những phần ký tự trong text đã khớp, giữ nguyên thứ tự gốc.
- **Thách thức:**
  - Bản thân fuzzy matching cho phép **bỏ qua** ký tự (gap), **không phân biệt hoa thường, tiếng Việt có dấu**.
  - Hiển thị highlight phải "thân thiện" (thẻ `<mark>`/`<span class="hl">`) và **không vỡ HTML**.

### 2. Khung giải pháp hai giai đoạn



### 3. Phase 1 – Tính toán vị trí khớp

Có nhiều thuật toán fuzzy; ở đây trình bày "**Scoped subsequence**" (đơn giản,  $O(n \cdot m)$ ), sau đó nâng cấp với **Bitap/Shift-Or** và thư viện.

#### 3.1. Thuật toán subsequence đơn giản

```
function fuzzyMatchIndexes(text, query) {
 const idxs = [];
 let ti = 0, qi = 0;

 const t = text.toLowerCase();
 const q = query.toLowerCase();

 while (ti < t.length && qi < q.length) {
 if (t[ti] === q[qi]) {
 idxs.push(ti); // lưu vị trí khớp
 qi++; // tiến ký tự query
 }
 ti++; // luôn tiến text
 }
}
```

```
// thành công khi đã "tiêu thụ" hết query
return qi === q.length ? idxs : null;
}
```

- Ý tưởng: query là *subsequence* của text (không cần liên tục).
- Phù hợp autocomplete: gõ "mh" khớp "My House".
- Trả về mảng chỉ số (``...) hoặc null nếu không khớp.

### 3.2. Cải tiến – Bitap (Shift-Or)

- *Bitap algorithm* dùng bitmask 64-bit; tìm subsequence trong  $O(n)$  với bit thao tác.
- JavaScript 2020 hỗ trợ BigInt  $\Rightarrow$  có thể tự cài hoặc **dùng thư viện**: fuse.js, fast-fuzzy, miniSearch.
- Các lib trả cả **score & indices**:

```
const fuse = new Fuse(list, { includeMatches: true });
const res = fuse.search(query);
// res[0].matches[0].indices = [[5, 7], ...]
```

## 4. Phase 2 – Sinh HTML highlight

Giả sử ta đã có idxs (mảng vị trí đơn lẻ) hoặc ranges (cặp [start,end]).

```
function highlight(text, idxs, tag = 'mark') {
 if (!idxs) return text;

 // Gom vị trí liên tiếp thành range [l,r]
 const ranges = [];
 let prev = -2, range = [];
 idxs.forEach(i => {
 if (i === prev + 1) { // tiếp nối
 range[1] = i; // mở rộng
 } else { // đứt đoạn
 if (range.length) ranges.push(range);
 range = [i, i];
 }
 prev = i;
 });
 if (range.length) ranges.push(range);

 // Sinh HTML
 let res = '';
 let last = 0;
 ranges.forEach(([l, r]) => {
 res += escapeHTML(text.slice(last, l)); // phần không highlight
 res += `<${tag}>` + escapeHTML(text.slice(l, r + 1)) + `</${tag}>`;
 last = r + 1;
 });
 res += escapeHTML(text.slice(last));
 return res;
}
```

```

}

function escapeHTML(s) {
 return s.replace(/&<>"/g, c => ({ '&': '&', '<': '<', '>': '>', '"': '"' }[c]))
}

```

- **Gộp range** để tránh lồng `<mark>` mỗi ký tự.
- `EscapeHTML` ngăn XSS khi text chứa `<`.

## 5. Ví dụ dummy end-to-end

```

const text = 'Ngân hàng Thương mại Cổ phần Kỹ thương Việt Nam'; // Techcombank
const query = 'thcVn'; // người dùng gõ tắt

const idxs = fuzzyMatchIndexes(text, query); // [11,12,13,38,40]
const html = highlight(text, idxs);

document.body.innerHTML = html;
/*
Kết quả hiển thị (đã bôi vàng):
Ngân hàng <mark>Thc</mark>ương mại Cổ phần Kỹ thương <mark>V</mark>iệt <mark>N</mark>am
*/

```

## 6. Nâng cao & tối ưu

### 1. Xử lý Tiếng Việt có dấu

- Chuẩn hoá bằng `String.prototype.normalize('NFD')` + remove `\p{Diacritic}` (RegExp `\p{M}`) trước khi match, nhưng highlight dựa trên **string gốc** bằng cách giữ map chỉ số.

### 2. Highlight nhiều field

- Với `Fuse.js` (`includeMatches`), mỗi match có `indices`; chỉ cần lặp và highlight từng field.

### 3. Hiệu năng list lớn

- Tiền xử lý từng text: build **"lowercase + no-diacritics" cache** để tránh normalize mỗi lần.
- Debounce input 200 ms.
- Web Worker cho tính toán nặng (10k+ dòng).

### 4. Server-side rendering

- Sử dụng cùng thuật toán để render sẵn `<mark>` rồi gửi HTML; giảm JavaScript ở client.

### 5. Accessibility

- Thay `<mark>` bằng `<span class="hl" role="mark">`, thêm `aria-label` khi màu nền khó thấy.

### 6. Themeable

```

.hl { background:#fff49d; color:inherit; }
@media (prefers-color-scheme:dark){

```

```
.hl { background:#665c00; }
}
```

## 7. Cheat-sheet triển khai nhanh

```
// 1□ khớp → idxs = fuzzyMatchIndexes(text, query)
// 2□ tô sáng → html = highlight(text, idxs)
// 3□ gán DOM → element.innerHTML = html;
```

## 8. Kết luận

- **Tách 2 phase** (tính match vs. render) giúp linh hoạt thay đổi thuật toán fuzzy mà không đụng phần UI.
- Thuật toán subsequence đủ cho autocomplete; Bitap/ thư viện nâng cao nếu cần ranking.
- Luôn **escape HTML** trước khi chèn; nhớ tối ưu hiệu năng và hỗ trợ đa ngôn ngữ.

✱✱

# HTTPS & “Man-in-the-Middle” (MITM) Attack

## 1. MITM là gì?

MITM xảy ra khi **kẻ tấn công chen giữa** kênh giao tiếp (client ⇌ server) để:

- Nghe lén (eavesdrop).
- Giả mạo, chỉnh sửa gói tin (tamper).
- Cướp phiên (session hijack).

Với **HTTP thuần**, dữ liệu gửi đi dạng *plaintext*, MITM chỉ cần:

- Đọc các gói TCP (sniff).
- Thay đổi nội dung rồi forward.

HTTPS (HTTP + TLS) **khắc phục** bằng:

1. **Mã hoá (encryption)** – nội dung được “bọc” AES, kẻ ngoài không đọc được.
2. **Xác thực (authentication)** – chứng chỉ số chứng minh server “là thật”.
3. **Toàn vẹn (integrity)** – HMAC/AEAD bảo đảm gói tin chưa bị sửa.

MITM tấn công HTTPS thế nào? → Phải **phá vỡ/né** các cơ chế trên.

## 2. Đường tấn công điển hình

## 2.1. Giáng cấp (Downgrade) kết nối

### 1. Strip HTTPS → HTTP

- Kẻ tấn công chặn yêu cầu `http://example.com` và xoá/đổi hướng 301 sang HTTPS.
- Nạn nhân vẫn ở HTTP không mã hoá → MITM đọc/chỉnh sửa tùy ý.
- Phòng thủ: **HSTS** (Strict-Transport-Security) + preload list trên trình duyệt.

### 2. TLS Version/Algo Downgrade

- Ép client rơi xuống TLS 1.0/RC4 dễ bẻ.
- Phòng thủ: `TLS_FALLBACK_SCSV`, chỉ hỗ trợ TLS 1.2+.

## 2.2. Giả mạo chứng chỉ (Certificate Spoofing)

### 1. CA bị hack hoặc cấp sai

- DigiNotar (2011) cấp nhầm cert `*.google.com`.
- MITM lắp cert giả nhưng **được trình duyệt tin** ⇒ Decrypt hoàn toàn.
- Phòng thủ:
  - **CT (Certificate Transparency)** – log mọi cert công khai, trình duyệt so khớp.
  - **Multi-Perspectives validation** (e.g. Chrome SST).

### 2. Cài CA gốc giả trên máy nạn nhân

- Malware thêm root CA, proxy công ty cài “SSL inspection”.
- Phòng thủ:
  - Bật **Certificate Pinning** (mobile app), HPKP (đã deprecated vì DoS risk).
  - Giám sát hệ điều hành → phát hiện CA lạ.

## 2.3. TLS Renegotiation & Session Hijack

- Chèn dữ liệu trước handshake (`prefix attack`) – đã vá bằng RFC 5746.
- Cướp **session cookie** qua sub-domain không HTTPS → “cookie forcing”.  
Phòng thủ: `Secure`; `SameSite`; `HttpOnly`.

## 2.4. DNS-based MITM

- **DNS spoofing / poisoning** trỏ `bank.com` → IP ác.
- Khi server ác dùng **Let's Encrypt wildcard** thì TLS vẫn hợp lệ.
- Phòng thủ:
  - **DNSSEC** (ký số bản ghi).
  - **DoH/DoT** giữa client & resolver.
  - **Pre-connect list** (Chrome “expect-CT”).

## 2.5. BGP Hijacking (Routing-level)

- Tấn công định tuyến quốc tế để “bè lái” prefix IP.
- Phòng thủ: **RPKI**, **QUIC zero-RTT resumption** giảm thời gian lộ route.

## 3. Quy trình MITM “classic” minh hoạ

1. Nạn nhân ở Wi-Fi công cộng.
2. MITM vận hành access-point giả “FreeAirportWiFi”.
3. Nạn nhân truy cập `https://gmail.com`.
4. MITM trả về **cert tự ký** → trình duyệt **cảnh báo**.
  - Nếu người dùng “Proceed anyway” → MITM thắng.
5. Trong trường hợp CA bị cài đặt sẵn (trong máy công ty) cảnh báo **không xuất hiện**.

## 4. Demo code dummy (Node.js)

```
// MITM proxy (ONLY for educational purpose)
const Proxy = require('http-mitm-proxy');
const fs = require('fs');
const proxy = Proxy();

proxy.onError = (ctx, err) => console.error('Proxy error', err);

proxy.onConnect((req, clientSocket, head, callback) => {
 // fake handshake: always "trust" any host
 return callback(); // continue TLS MITM
});

proxy.onRequest((ctx, callback) => {
 console.log('→', ctx.clientToProxyRequest.url);
 ctx.onResponseData((ctx, chunk, cb) => {
 // Modify response here ...
 cb(null, chunk);
 });
 return callback();
});

proxy.listen({ port: 8000 });
console.log('△ MITM proxy listening 127.0.0.1:8000');
```

- Tool như *mitmproxy*, *Charles* làm tương tự: tạo CA tự ký, yêu cầu cài vào trusted store.

## 5. Phòng thủ đa tầng (Best Practices)

Front-end / App:

- Luôn dùng **absolute HTTPS** cho mọi asset; cấm `http://`.
- Bật **HSTS** (`max-age >= 6 tháng; includeSubDomains; preload`).
- Content-Security-Policy: `upgrade-insecure-requests`.

- Cookie: Secure; SameSite=Lax; HttpOnly;

Backend:

- Chỉ cho TLS 1.2/1.3, ciphersuites mạnh (AES-GCM, ChaCha20-Poly1305).
- OCSP Stapling + Must-Staple → tránh revocation fail-open.
- Theo dõi CT log, bật metapolicies: **expect-CT** (đang dần thay bằng PKP-RO).

Network / Org:

- DNSSEC trên zone, hỗ trợ DoT/DoH ở resolver.
- RPKI ký prefix.
- Zero-Trust: Mutual TLS (mTLS) nội bộ.

Client-side advanced:

- **Certificate Pinning** (mobile app): SHA-256 SPKI.
- Use **DNS-over-HTTPS & Encrypted SNI (ESNI/ECH)** để ẩn SNI.
- Use password-less (WebAuthn) tránh credential sniff.

## 6. Takeaways nhanh

1. HTTPS **không “an toàn tuyệt đối”** – phụ thuộc chain CA, DNS, route.
2. MITM hiện đại tập trung vào **hạ tầng PKI**: lạm dụng CA, cài root CA nội bộ, DNS/BGP hijack.
3. Phòng thủ phải **đa lớp**: HSTS, CT, pinning, DNSSEC, RPKI, mTLS.
4. User awareness quan trọng: **ĐỪNG “bypass” cảnh báo certificate**.
5. Làm FE/BE, bạn nên:
  - Bật toàn bộ header bảo mật.
  - Tự động hoá giám sát cert, gia hạn, CT log.
  - Test downgrade / strip bằng công cụ như `ssllabs`, `zyan.green/openssl`.

## Bài 92 – Truy vết cha (con đường ID) trong cây JSON phẳng

Giả sử ta có “bảng” dữ liệu dạng list (mỗi node chứa `id`, `parentId`).

Nhiệm vụ: với `value = '112'` hãy sinh hàm `fn` trả về mảng tất cả **ID tổ tiên** từ gốc đến node này:  
`fn('112') → .`

### 1. Phân tích bài toán

1. Ta chỉ cần **một con đường duy nhất** (không phải toàn bộ cây).
2. Dữ liệu thường ở dạng **hash map** (`id → node`) ⇒ tra cứu  $O(1)$ .
3. Thuật toán:
  - Bước 1 – tạo “sổ cái” map cho toàn bộ list.

- Bước 2 – từ value lần lượt đi ngược parentId cho tới khi gặp gốc (parentId === 0 hoặc undefined).
- Bước 3 – đảo ngược kết quả (vì ta đi từ lá → gốc) hoặc “unshift” từng bước.

Độ phức tạp:

- Time  $O(h)$  –  $h$  là chiều cao (không quá  $n$ ).
- Space  $O(h)$  cho mảng kết quả (map tốn  $O(n)$ , có thể tái dùng).

## 2. Cài đặt hàm fn

```
// Demo dataset: id, name, parentId
const list = [
 { id: 1, name: '广东省', parentId: 0 },
 { id: 11, name: '深圳市', parentId: 1 },
 { id: 111, name: '南山区', parentId: 11 },
 { id: 112, name: '福田区', parentId: 11 }
];

/**
 * Tìm chuỗi ancestor của một node.
 * @param {string|number} value id cần tìm
 * @param {Array<Object>} data danh sách phẳng
 * @param {string} [idKey='id'] tên trường id
 * @param {string} [pidKey='parentId'] tên trường parentId
 * @returns {Array<number>} mảng id từ gốc → node
 */
function getAncestors(value, data, idKey = 'id', pidKey = 'parentId') {
 // ❶ Dựng hash map id → node
 const map = new Map();
 for (const item of data) map.set(item[idKey], item);

 // ❷ Lần ngược lên cha
 const path = [];
 let currentId = Number(value); // ép về number cho chắc
 while (map.has(currentId)) {
 path.push(currentId);
 const parentId = map.get(currentId)[pidKey];
 if (parentId === 0 || parentId === undefined) break;
 currentId = parentId;
 }

 // ❸ Đảo ngược để được gốc → lá
 return path.reverse();
}

// API như đề
const fn = (value) => getAncestors(value, list);

console.log(fn(112)); // [1, 11, 112]
```

### 3. Giải thích chi tiết

1. **Hash map** giúp ta không phải quét cả mảng mỗi bước:  
`id → { id, parentId, ... }`  
Tra cứu  $O(1)$ , tổng cộng chỉ bằng số tổ tiên (thường rất ít).
2. **Loop while:**
  - Ghi lại `currentId` vào `path`.
  - Lấy cha: `parentId = map.get(currentId).parentId`.
  - Nếu cha bằng 0 (hoặc null/undefined theo hệ thống) thì dừng.
3. **reverse()** cuối cùng để kết quả đúng thứ tự gốc → lá.

### 4. Mở rộng nâng cao

1. **Tái sử dụng map**  
Nếu nhiều lần gọi `fn`, hãy xây dựng map một lần rồi truyền vào (hoặc đóng gói trong closure) để tiết kiệm  $O(n)$  mỗi lần.
2. **Phát hiện chu kỳ**  
Nếu dữ liệu sai (`parentId` lặp), thêm Set `visited` để break và cảnh báo.
3. **Trả về object đầy đủ**  
Sửa `path.push(item)` để nhận được mảng node thay vì chỉ `id`.
4. **TypeScript generic**

```
type Node<T extends number|string = number> = {
 id: T;
 parentId: T | 0;
 [k: string]: unknown;
};
function ancestors<T extends Node>(
 id: T['id'], list: T[]
): T['id'][] { ... }
```

### 5. IndexedDB / SQL

Trong DB có thể dùng truy vấn đệ quy (CTE) `WITH RECURSIVE` để lấy path.

Như vậy, chỉ với 2 vòng lặp ngắn (xây map + truy vết), ta tìm được **đường cha** nhanh gọn, đúng nguyên lý "tra cứu  $O(1)$  – tối thiểu hoá độ phức tạp".

✱

## BÀI 93 – "MEDIAN OF TWO SORTED ARRAYS" – Bài toán kinh điển Binary Search

## I. PHÂN TÍCH BÀI TOÁN

**Đầu vào:** Hai mảng đã sắp xếp `nums1[m]`, `nums2[n]`

**Đầu ra:** Median của mảng hợp nhất (không thực sự merge)

**Ràng buộc:** Time complexity  **$O(\log(m+n))$**  - loại bỏ merge  $O(m+n)$

**Thách thức chính:**

- Không được merge thật sự (sẽ là  $O(m+n)$ )
- Phải "tưởng tượng" mảng merged để tìm vị trí giữa
- Xử lý cả trường hợp độ dài chẵn/lẻ

## II. INTUITION & APPROACH EVOLUTION

### 1. Naive Approach – $O(m+n)$ [KHÔNG đạt yêu cầu]

```
function findMedianNaive(nums1, nums2) {
 const merged = [];
 let i = 0, j = 0;

 // Merge two sorted arrays
 while (i < nums1.length && j < nums2.length) {
 if (nums1[i] <= nums2[j]) {
 merged.push(nums1[i++]);
 } else {
 merged.push(nums2[j++]);
 }
 }

 // Add remaining elements
 while (i < nums1.length) merged.push(nums1[i++]);
 while (j < nums2.length) merged.push(nums2[j++]);

 const n = merged.length;
 return n % 2 === 0
 ? (merged[n/2-1] + merged[n/2]) / 2
 : merged[Math.floor(n/2)];
}
```

### 2. Optimal Binary Search Approach – $O(\log(\min(m,n)))$

**Ý tưởng cốt lõi:**

- Thay vì merge, ta tìm **điểm cắt (partition)** trên cả hai mảng
- Đảm bảo phần bên trái có  **$(m+n+1)/2$**  phần tử
- Phần tử lớn nhất bên trái  $\leq$  phần tử nhỏ nhất bên phải

```
function findMedianSortedArrays(nums1, nums2) {
 // Đảm bảo nums1 là mảng ngắn hơn để tối ưu binary search
 if (nums1.length > nums2.length) {
```

```

 [nums1, nums2] = [nums2, nums1];
 }

 const m = nums1.length;
 const n = nums2.length;
 const totalLeft = Math.floor((m + n + 1) / 2);

 let left = 0, right = m;

 while (left <= right) {
 // Partition positions
 const cut1 = Math.floor((left + right) / 2); // Cut in nums1
 const cut2 = totalLeft - cut1; // Cut in nums2

 // Elements around cuts
 const left1 = cut1 === 0 ? -Infinity : nums1[cut1 - 1];
 const left2 = cut2 === 0 ? -Infinity : nums2[cut2 - 1];
 const right1 = cut1 === m ? Infinity : nums1[cut1];
 const right2 = cut2 === n ? Infinity : nums2[cut2];

 // Check if partition is correct
 if (left1 <= right2 && left2 <= right1) {
 // Found the correct partition
 if ((m + n) % 2 === 0) {
 // Even total length
 return (Math.max(left1, left2) + Math.min(right1, right2)) / 2;
 } else {
 // Odd total length
 return Math.max(left1, left2);
 }
 } else if (left1 > right2) {
 // nums1 cut is too far right, move left
 right = cut1 - 1;
 } else {
 // nums1 cut is too far left, move right
 left = cut1 + 1;
 }
 }

 throw new Error("Input arrays are not sorted or invalid");
}

```

### III. GIẢI THÍCH CHI TIẾT THUẬT TOÁN

#### 3.1. Visualization của Partition Strategy

```

nums1: [1, 3, 8, 9, 15] m = 5
nums2: [7, 11, 18, 19, 21, 25] n = 6

Total elements = 11 (odd)
Left partition should have (11+1)/2 = 6 elements

Try cut1 = 2 (take first 2 from nums1):
nums1: [1, 3] | [8, 9, 15]

```

```
nums2: [7, 11, 18, 19] | [21, 25] (take first 4 from nums2)
```

Check conditions:

- left1 = 3, right2 = 21 →  $3 \leq 21$  ✓

- left2 = 19, right1 = 8 →  $19 \leq 8$  ✗

left2 > right1 → nums1 cut too far left, move right

### 3.2. Step-by-step walkthrough

```
// Example: nums1 = [1, 2], nums2 = [3, 4]
```

```
/*
```

```
m=2, n=2, totalLeft = (2+2+1)/2 = 2
```

Iteration 1:

left=0, right=2, cut1=1, cut2=1

left1=1, left2=3, right1=2, right2=4

Check:  $1 \leq 4$  ✓,  $3 \leq 2$  ✗ → left2 > right1 → left = cut1+1 = 2

Iteration 2:

left=2, right=2, cut1=2, cut2=0

left1=2, left2=-∞, right1=∞, right2=3

Check:  $2 \leq 3$  ✓,  $-\infty \leq \infty$  ✓ → Found!

Total even → median =  $(\max(2, -\infty) + \min(\infty, 3))/2 = (2+3)/2 = 2.5$

```
*/
```

## IV. COMPLEXITY ANALYSIS

| Aspect            | Naive Merge | Binary Search        |
|-------------------|-------------|----------------------|
| <b>Time</b>       | $O(m + n)$  | $O(\log(\min(m,n)))$ |
| <b>Space</b>      | $O(m + n)$  | $O(1)$               |
| <b>Best Case</b>  | $O(m + n)$  | $O(1)$               |
| <b>Worst Case</b> | $O(m + n)$  | $O(\log(\min(m,n)))$ |

### Tại sao $O(\log(\min(m,n)))$ ?

- Binary search trên mảng ngắn hơn
- Mỗi iteration giảm search space đi một nửa
- Tối đa  $\log(\min(m,n))$  iterations

## V. EDGE CASES VÀ XỬ LÝ

```
function findMedianRobust(nums1, nums2) {
 // Input validation
 if (!Array.isArray(nums1) || !Array.isArray(nums2)) {
 throw new TypeError("Both inputs must be arrays");
 }
}
```

```

// Handle empty arrays
if (nums1.length === 0 && nums2.length === 0) {
 throw new Error("Both arrays cannot be empty");
}

if (nums1.length === 0) {
 const n = nums2.length;
 return n % 2 === 0
 ? (nums2[n/2-1] + nums2[n/2]) / 2
 : nums2[Math.floor(n/2)];
}

if (nums2.length === 0) {
 const m = nums1.length;
 return m % 2 === 0
 ? (nums1[m/2-1] + nums1[m/2]) / 2
 : nums1[Math.floor(m/2)];
}

// Ensure nums1 is shorter for optimization
if (nums1.length > nums2.length) {
 return findMedianRobust(nums2, nums1);
}

const m = nums1.length;
const n = nums2.length;
const totalLeft = Math.floor((m + n + 1) / 2);

let left = 0, right = m;

while (left <= right) {
 const cut1 = Math.floor((left + right) / 2);
 const cut2 = totalLeft - cut1;

 // Boundary checks
 if (cut2 < 0) {
 right = cut1 - 1;
 continue;
 }
 if (cut2 > n) {
 left = cut1 + 1;
 continue;
 }

 const left1 = cut1 === 0 ? -Infinity : nums1[cut1 - 1];
 const left2 = cut2 === 0 ? -Infinity : nums2[cut2 - 1];
 const right1 = cut1 === m ? Infinity : nums1[cut1];
 const right2 = cut2 === n ? Infinity : nums2[cut2];

 if (left1 <= right2 && left2 <= right1) {
 if ((m + n) % 2 === 0) {
 return (Math.max(left1, left2) + Math.min(right1, right2)) / 2;
 } else {
 return Math.max(left1, left2);
 }
 }
}

```

```

 } else if (left1 > right2) {
 right = cut1 - 1;
 } else {
 left = cut1 + 1;
 }
}

throw new Error("Invalid input - arrays might not be sorted");
}

```

## VI. TEST CASES COMPREHENSIVE

```

// Test suite
const testCases = [
 // Basic cases
 { nums1: [1, 3], nums2: [^22_2], expected: 2.0 },
 { nums1: [1, 2], nums2: [3, 4], expected: 2.5 },

 // Edge cases
 { nums1: [], nums2: [^22_1], expected: 1.0 },
 { nums1: [^22_2], nums2: [], expected: 2.0 },
 { nums1: [^22_1], nums2: [^22_2], expected: 1.5 },

 // Large arrays
 { nums1: [1, 3, 8, 9, 15], nums2: [7, 11, 18, 19, 21, 25], expected: 11.0 },

 // All elements in one array smaller
 { nums1: [1, 2, 3], nums2: [4, 5, 6], expected: 3.5 },

 // Interleaved
 { nums1: [1, 3, 5], nums2: [2, 4, 6], expected: 3.5 },

 // Duplicates
 { nums1: [1, 1, 1], nums2: [1, 1, 1], expected: 1.0 },

 // Negative numbers
 { nums1: [-5, -3, -1], nums2: [-2, 0, 2], expected: -1.5 }
];

testCases.forEach(({ nums1, nums2, expected }, i) => {
 const result = findMedianRobust(nums1, nums2);
 console.log(`Test ${i + 1}: ${result} === expected ? '✓' : '✗'`);
 console.log(`Expected: ${expected}, Got: ${result}`);
});

```

## VII. BIẾN THỂ VÀ MỞ RỘNG

## 7.1. Kth Element in Two Sorted Arrays

```
function findKthElement(nums1, nums2, k) {
 if (nums1.length > nums2.length) {
 return findKthElement(nums2, nums1, k);
 }

 const m = nums1.length;
 const n = nums2.length;

 if (k > m + n || k < 1) {
 throw new Error("K is out of range");
 }

 let left = Math.max(0, k - n);
 let right = Math.min(k, m);

 while (left <= right) {
 const cut1 = Math.floor((left + right) / 2);
 const cut2 = k - cut1;

 const left1 = cut1 === 0 ? -Infinity : nums1[cut1 - 1];
 const left2 = cut2 === 0 ? -Infinity : nums2[cut2 - 1];
 const right1 = cut1 === m ? Infinity : nums1[cut1];
 const right2 = cut2 === n ? Infinity : nums2[cut2];

 if (left1 <= right2 && left2 <= right1) {
 return Math.max(left1, left2);
 } else if (left1 > right2) {
 right = cut1 - 1;
 } else {
 left = cut1 + 1;
 }
 }

 throw new Error("Invalid input");
}

// Median using Kth element
function findMedianUsingKth(nums1, nums2) {
 const total = nums1.length + nums2.length;
 if (total % 2 === 1) {
 return findKthElement(nums1, nums2, Math.floor(total / 2) + 1);
 } else {
 const mid1 = findKthElement(nums1, nums2, total / 2);
 const mid2 = findKthElement(nums1, nums2, total / 2 + 1);
 return (mid1 + mid2) / 2;
 }
}
```

## 7.2. Multiple Sorted Arrays

```
function findMedianMultipleArrays(arrays) {
 // Reduce to binary problem
 while (arrays.length > 2) {
 const newArrays = [];
 for (let i = 0; i < arrays.length; i += 2) {
 if (i + 1 < arrays.length) {
 // Merge two arrays
 const merged = mergeTwoSorted(arrays[i], arrays[i + 1]);
 newArrays.push(merged);
 } else {
 newArrays.push(arrays[i]);
 }
 }
 arrays = newArrays;
 }

 if (arrays.length === 1) {
 const arr = arrays[0];
 const n = arr.length;
 return n % 2 === 0
 ? (arr[n/2-1] + arr[n/2]) / 2
 : arr[Math.floor(n/2)];
 } else {
 return findMedianRobust(arrays[0], arrays[1]);
 }
}
```

## 7.3. Streaming Median (Bonus)

```
class MedianFinder {
 constructor() {
 this.maxHeap = new MaxHeap(); // left half
 this.minHeap = new MinHeap(); // right half
 }

 addNum(num) {
 // Always add to maxHeap first
 this.maxHeap.push(num);

 // Balance: ensure maxHeap.top() <= minHeap.top()
 if (this.minHeap.size() > 0 &&
 this.maxHeap.top() > this.minHeap.top()) {
 this.minHeap.push(this.maxHeap.pop());
 }

 // Balance sizes: |maxHeap| - |minHeap| ∈ {0, 1}
 if (this.maxHeap.size() > this.minHeap.size() + 1) {
 this.minHeap.push(this.maxHeap.pop());
 } else if (this.minHeap.size() > this.maxHeap.size() + 1) {
 this.maxHeap.push(this.minHeap.pop());
 }
 }
}
```

```

findMedian() {
 if (this.maxHeap.size() === this.minHeap.size()) {
 return (this.maxHeap.top() + this.minHeap.top()) / 2;
 } else if (this.maxHeap.size() > this.minHeap.size()) {
 return this.maxHeap.top();
 } else {
 return this.minHeap.top();
 }
}
}
}

```

## VIII. PERFORMANCE BENCHMARKING

```

function benchmarkMedianAlgorithms() {
 const sizes = [100, 1000, 10000, 100000];

 sizes.forEach(size => {
 const nums1 = Array.from({length: size/2}, (_, i) => i * 2);
 const nums2 = Array.from({length: size/2}, (_, i) => i * 2 + 1);

 // Benchmark naive approach
 console.time(`Naive-${size}`);
 for (let i = 0; i < 100; i++) {
 findMedianNaive([...nums1], [...nums2]);
 }
 console.timeEnd(`Naive-${size}`);

 // Benchmark binary search
 console.time(`Binary-${size}`);
 for (let i = 0; i < 100; i++) {
 findMedianRobust(nums1, nums2);
 }
 console.timeEnd(`Binary-${size}`);
 });
}

// benchmarkMedianAlgorithms();

```

## IX. REAL-WORLD APPLICATIONS

### 9.1. Database Query Optimization

```

-- Tương đương SQL (PostgreSQL)
WITH ranked AS (
 SELECT value,
 ROW_NUMBER() OVER (ORDER BY value) as rn,
 COUNT(*) OVER () as total
 FROM (
 SELECT value FROM table1
 UNION ALL
 SELECT value FROM table2
)
)

```

```

) combined
)
 SELECT AVG(value::NUMERIC) as median
 FROM ranked
 WHERE rn IN (
 CEIL(total/2.0),
 FLOOR(total/2.0) + 1
);

```

## 9.2. Real-time Analytics

```

class RealTimeMedianTracker {
 constructor() {
 this.sortedArrays = [];
 this.totalCount = 0;
 }

 addDataSource(sortedArray) {
 this.sortedArrays.push(sortedArray);
 this.totalCount += sortedArray.length;
 }

 getCurrentMedian() {
 if (this.sortedArrays.length === 0) return null;
 if (this.sortedArrays.length === 1) {
 const arr = this.sortedArrays[0];
 const n = arr.length;
 return n % 2 === 0
 ? (arr[n/2-1] + arr[n/2]) / 2
 : arr[Math.floor(n/2)];
 }

 // For multiple arrays, use divide & conquer
 return this.findMedianRecursive(this.sortedArrays);
 }

 findMedianRecursive(arrays) {
 if (arrays.length === 1) {
 const arr = arrays[0];
 return arr.length % 2 === 0
 ? (arr[arr.length/2-1] + arr[arr.length/2]) / 2
 : arr[Math.floor(arr.length/2)];
 }

 if (arrays.length === 2) {
 return findMedianRobust(arrays[0], arrays[1]);
 }

 // Divide and conquer
 const mid = Math.floor(arrays.length / 2);
 const left = arrays.slice(0, mid);
 const right = arrays.slice(mid);

 const leftMedian = this.findMedianRecursive(left);
 const rightMedian = this.findMedianRecursive(right);

```

```
 return (leftMedian + rightMedian) / 2; // Approximation
}
}
```

## X. KẾT LUẬN VÀ TAKEAWAYS

### Key Points:

1. **Binary Search trên partition thay vì elements** - insight chính
2.  **$O(\log(\min(m,n)))$**  đạt được bằng cách search trên mảng ngắn hơn
3. **Partition strategy** đảm bảo  $\text{left\_max} \leq \text{right\_min}$
4. **Edge cases** rất quan trọng: empty arrays, unequal lengths

### Interview Tips:

- Bắt đầu với naive solution để thể hiện understanding
- Giải thích tại sao  $O(\log(m+n))$  là challenge
- Draw partition visualization
- Handle edge cases carefully
- Discuss follow-up questions (Kth element, multiple arrays)

### Common Mistakes:

- Không handle empty arrays
- Partition logic sai (off-by-one errors)
- Không optimize binary search trên mảng ngắn hơn
- Integer overflow với large numbers

**Pattern này** là nền tảng cho:

- Binary search on answer
- Partition-based algorithms
- Divide & conquer on sorted structures
- Order statistics problems

Hiểu sâu "Median of Two Sorted Arrays" giúp master binary search techniques và solve các bài toán tương tự như "Kth smallest element", "Search in rotated array", và "Time-based key-value store".</answer>

## Bài 95 – Tự viết hàm deepClone “xịn”

Mục tiêu: tạo bản sao **đệ quy** của mọi kiểu dữ liệu JS thường gặp, hỗ trợ

1. Đối tượng, mảng, Map, Set, Date, RegExp, TypedArray...
2. Sao chép **Symbol-key & Symbol value**.
3. Phát hiện **tham chiếu vòng** (circular reference) và tái sử dụng node đã clone.

### 1. Vấn đề của `JSON.parse(JSON.stringify())`

- Bỏ qua function, undefined, Symbol, BigInt.
- Mất Date → string, RegExp → {}, prototype biến mất.
- Crash khi gặp vòng tham chiếu.

### 2. Thiết kế giải pháp

1. Duyệt DFS: khi gặp value cần clone,
  - Nếu là **primitive** (number, string, boolean, null, undefined, Symbol, BigInt) → trả thẳng.
  - Nếu là **Function** → giữ nguyên reference (hoặc clone mã nguồn tùy yêu cầu).
  - Nếu là **object**:
    - a. Kiểm tra **WeakMap cache**: đã copy chưa → trả bản copy để xử lý vòng lặp.
    - b. Tạo “shell” phù hợp (`new obj.constructor()` hoặc đặc biệt cho Map, Set...).
    - c. Lặp **cả key string lẫn key Symbol** với `Reflect.ownKeys()`.
    - d. Gán thuộc tính, tiếp tục đệ quy.
2. Sao chép property descriptor & prototype?
  - Để đơn giản, chỉ **data property** enumerable - đủ 90% use-case.
  - Có thể dùng `Object.getOwnPropertyDescriptors()` + `Object.create(proto, desc)` để giữ getter/setter.
3. TypedArray & ArrayBuffer: clone bằng `value.slice(0)` hoặc `value.buffer.slice(0)`.
4. Map/Set: lặp entry, clone key lẫn value.

### 3. Cài đặt

```
function deepClone(source, cache = new WeakMap()) {
 // 1. Primitive hoặc function => trả luôn
 if (Object(source) !== source || source instanceof Function) return source;

 // 2. Tránh lặp vô hạn
 if (cache.has(source)) return cache.get(source);

 // 3. Xử lý các lớp đặc biệt
 const type = Object.prototype.toString.call(source).slice(8, -1); // "[object Map]" ->
 let target;

 switch (type) {
 case 'Date':
 target = new Date(source.getTime());
```

```

 break;
 case 'RegExp':
 target = new RegExp(source.source, source.flags);
 target.lastIndex = source.lastIndex;
 break;
 case 'Map':
 target = new Map();
 cache.set(source, target); // set sớm để handle self-ref trong key/value
 source.forEach((v, k) => {
 target.set(deepClone(k, cache), deepClone(v, cache));
 });
 return target;
 case 'Set':
 target = new Set();
 cache.set(source, target);
 source.forEach(v => {
 target.add(deepClone(v, cache));
 });
 return target;
 case 'Array':
 target = new Array(source.length);
 break;
 case 'ArrayBuffer':
 target = source.slice(0);
 break;
 case 'Uint8Array':
 case 'Uint16Array':
 case 'Uint32Array':
 case 'Int8Array':
 case 'Int16Array':
 case 'Int32Array':
 case 'Float32Array':
 case 'Float64Array':
 case 'BigInt64Array':
 case 'BigUint64Array':
 target = new source.constructor(source); // clone underlying buffer
 break;
 default:
 target = Object.create(Object.getPrototypeOf(source));
}

// 4. Lưu vào cache
cache.set(source, target);

// 5. Sao chép thuộc tính (bao gồm Symbol)
Reflect.ownKeys(source).forEach(key => {
 const desc = Object.getOwnPropertyDescriptor(source, key);
 if (desc.get || desc.set) {
 Object.defineProperty(target, key, desc); // giữ nguyên accessor
 } else {
 desc.value = deepClone(source[key], cache); // đệ quy
 Object.defineProperty(target, key, desc);
 }
});

```

```
 return target;
 }
}
```

## 4. Thử nghiệm “dummy”

```
const symKey = Symbol('meta');
const circle = { num: 1 };
circle.self = circle;

const original = {
 a: 10,
 b: new Date(),
 c: /ab+c/gi,
 d: new Map([[{ x: 1 }, ['deep', { y: 2 }]]]),
 e: new Set([1, 2, 3]),
 f: new Uint8Array([7, 8, 9]),
 [symKey]: 'symbolValue'
};
original.circle = circle;

const cloned = deepClone(original);

console.log(cloned);
console.log(cloned.d.keys().next().value === [...original.d.keys()][0]); // false
console.log(cloned[symKey]); // "symbolValue"
console.log(cloned.circle === cloned.circle.self); // true
console.log(cloned.circle === original.circle); // false
```

Kết quả:

- Thuộc tính Symbol giữ nguyên.
- Map/Set, TypedArray, Date, RegExp clone đúng.
- Vòng tham chiếu được tái lập mà **không** gây tràn ngăn xếp.

## 5. Mở rộng

1. **Shallow vs Deep** chọn lọc: truyền danh sách key cần deep, còn lại shallow (kỹ thuật copy-on-write).
2. **Structured Clone** (browser/Node 17+): `structuredClone(obj)` đã sẵn hỗ trợ Symbol key & circular, ngoại trừ function/DOM node – dùng khi môi trường cho phép.
3. Backup hiệu suất: với object rất lớn, clone theo **bộ** (batch) trong Web Worker để tránh block UI.
4. Immutable.js / immer: tránh clone tốn bộ nhớ nhờ persistent data structure.

### Ghi nhớ nhanh:

- Dùng **WeakMap** chống vòng tham chiếu.
- Lặp qua key bằng `Reflect.ownKeys` để không bỏ sót Symbol.
- Đặc biệt hoá **Date, RegExp, Map, Set, TypedArray**; các kiểu còn lại xử lý chung.

# Những đoạn “cần mã hoá” trên front-end được đặt ở đâu?

## 1. Truyền dữ liệu qua mạng

- Đăng nhập (password/OTP).
- Thông tin nhạy cảm (thẻ tín dụng, hồ sơ y tế, token OAuth).
- Tài xuống tệp giấy phép, file cấu hình bí mật.

## 2. Lưu trữ cục bộ

- Access/Refresh-token trong LocalStorage/IndexedDB.
- Cơ sở dữ liệu offline (PWA, Notes, Password manager).
- File được người dùng mã hoá trước khi upload (“client-side encryption”).

## 3. Mã hoá logic kinh doanh

- Chữ ký số request (AWS SigV4, Shopify HMAC).
- Watermark DRM, phát video HLS + CENC.
- Ẩn thuật toán/ khóa API trong SPA® (thực tế chỉ “obfuscate”, không bảo mật thật).

Front-end **chỉ nên coi là lớp bảo vệ bổ sung**, không thay thế bảo mật phía server vì khóa nằm ngay trong trình duyệt.

## 1. Mã hoá khi truyền dữ liệu

| Mục tiêu                             | Phương pháp                                      | Thư viện/browser API                                           |
|--------------------------------------|--------------------------------------------------|----------------------------------------------------------------|
| Kết nối kênh                         | <b>HTTPS/TLS</b>                                 | Mặc định <code>fetch</code> , <code>XMLHttpRequest</code>      |
| Bảo mật payload trước khi gửi (E2EE) | AES-GCM + RSA/OAEP hoặc X25519-ChaCha20-Poly1305 | Web Crypto API ( <code>window.crypto.subtle</code> )           |
| Chữ ký yêu cầu (request signing)     | HMAC-SHA256, Ed25519                             | <code>crypto.subtle.sign</code> hoặc <code>libsodium.js</code> |
| Thêm tầng bảo mật cho websocket      | <code>wss://</code> + DTLS-SRTP (WebRTC)         | built-in                                                       |

Ví dụ gửi mật khẩu E2EE:

```
// 1. Tạo cặp khoá dùng cho phiên (ECDH)
const {publicKey, privateKey} = await crypto.subtle.generateKey(
 {name: 'ECDH', namedCurve: 'P-256'}, true, ['deriveKey']
);
// 2. Nhận publicKey của server => derive shared secret
const sharedKey = await crypto.subtle.deriveKey(
 {name: 'ECDH', public: serverPub},
 privateKey,
 {name: 'AES-GCM', length: 256},
 false, ['encrypt']
);
// 3. Mã hoá password
const iv = crypto.getRandomValues(new Uint8Array(12));
```

```
const cipher = await crypto.subtle.encrypt(
 {name: 'AES-GCM', iv}, sharedKey,
 new TextEncoder().encode(password)
);
// Gửi {clientPub, iv, cipher} lên backend
```

## 2. Mã hoá khi lưu trữ offline

### a. Token & dữ liệu nhẹ

- **Web Crypto + IndexedDB:**

1. Sinh `masterKey` sau khi đăng nhập (dùng mật khẩu/biometrics để “unwrap”).
2. Mã hoá token rồi lưu.
3. Xoá `masterKey` khỏi RAM sau mỗi phiên.

### b. Database lớn (PWA)

- **sql.js + AES CBC:** ghi file `.sqlite` đã mã hoá, mở khoá khi user nhập pass-phrase.
- Sync bí mật: mỗi row có cột `ciphertext`, server chỉ là “bucket”.

### c. File upload mã hoá client-side (“End-to-End Cloud Storage”)

```
const fileBuf = await file.arrayBuffer();
const key = crypto.getRandomValues(new Uint8Array(32));
const iv = crypto.getRandomValues(new Uint8Array(12));

const cipher = await crypto.subtle.encrypt(
 {name: 'AES-GCM', iv}, await crypto.subtle.importKey(
 'raw', key, 'AES-GCM', false, ['encrypt']
), fileBuf
);
// Gửi cipher & iv, còn key chia sẻ qua kênh riêng (hoặc dùng RSA để wrap)
```

## 3. DRM & Multimedia

### 1. Encrypted Media Extensions (EME)

- Browser cấp “CDM” (Widevine, FairPlay).
- HLS/DASH chứa segment AES CTR; license server cấp key.

### 2. Subtle crypto watermark: SHA-256(video + token) → overlay.

### 3. WebAssembly + CENC cho ứng dụng React Native Desktop / Electron.

## 4. Obfuscation & Integrity

| Kỹ thuật                   | Công cụ                     | Ghi chú                                                         |
|----------------------------|-----------------------------|-----------------------------------------------------------------|
| Shrink/rename/garble       | Terser --mangle-props       | Giảm đọc hiểu, nhưng dựng lại dễ.                               |
| Runtime integrity check    | Subresource Integrity (SRI) | <code>&lt;script src="app.js" integrity="sha384-..."&gt;</code> |
| Code signing               | Trusted Types + CSP hash    | Ngăn XSS eval.                                                  |
| WASM blob + dynamic import | wasm-pack, Rust             | Ẩn logic, nhưng vẫn bị dump qua DevTools.                       |

## 5. Phòng ngừa vòng tham chiếu khoá (circular key "leak")

1. **Không nhét khoá bí mật** cố định vào JS bundle.
2. Tách key thành nhiều phần (Shamir Secret Sharing) + lấy 1 phần từ server sau đăng nhập.
3. Thời gian sống ngắn (rotate mỗi vài phút, refresh-token dùng MTLS).
4. Storage Isolation:
  - Chrome Storage Partitioning + cookie `SameSite=Strict; Secure; HttpOnly`.
  - ServiceWorker giữ key trong memory, không ghi LocalStorage.

## 6. Thư viện/mô-đun khuyến dùng

- `window.crypto.subtle` – AES-GCM, RSA-PSS, ECDH, HKDF.
- **libsodium-wasm** – high-level (sealed box, secretbox).
- **tweetnacl.js** – Ed25519 sign/verify, Xsalsa20-Poly1305.
- **openpgp.js** – PGP E2EE (mail, chat).
- **js-encrypt** – nhỏ gọn RSA.

## 7. Checklist "dummy" cho dự án SPA

1. Luôn `https://`; bật HSTS & SRI.
2. Token lưu trong **memory** hoặc **encrypted IndexedDB** (không LocalStorage plain).
3. Nếu cần E2EE:
  - Dùng ECDH trao khoá → AES-GCM.
  - Không dính khoá vào URL query.
4. WebSocket → `wss://` + optional JWT in header.
5. Chunk upload → mã hoá từng chunk, metadata (tag, MIME) đặt vào header được sign HMAC.
6. Bật CSP: `default-src 'self'; script-src 'self' 'sha256-...'`.
7. Kiểm toán bundle: không chứa chuỗi "PRIVATE KEY", "-----BEGIN".

8. Unit-test: decrypt ngay trong CI để bảo đảm mã hoá/giải mã tương thích backend.

## 8. Limitations & Lời khuyên

- **Front-end không thể giấu tuyệt đối khoá** khỏi kẻ khai thác DevTools—chỉ khó hơn.
- Mã hoá client-side **tăng CPU**; cân nhắc Web Worker.
- Đừng tự chế thuật toán; dùng chuẩn được kiểm chứng & thư viện audited.
- Luôn kết hợp **backend validation** (CSRF, rate-limit, token rotation) để hoàn thiện chuỗi bảo mật.

Tóm lại, front-end encryption xoay quanh **TLS + Web Crypto + lưu trữ an toàn + DRM**. Hãy coi trình duyệt là “môi trường bán tin cậy”, áp dụng mã hoá đúng chuẩn, xoá key khỏi persistent storage càng sớm càng tốt và luôn song hành với bảo mật phía server.

## Vì sao lại có $O(n^3)$ và $O(n)$ trong thuật toán “diff” của UI framework?

### 1. Bối cảnh: “diff” là gì?

Khi **cây ảo (Virtual DOM)** mới được tạo sau lần render, React/Vue phải so sánh (diff) với cây cũ để:

1. Phát hiện node nào thay đổi.
2. Sinh “patch” tối thiểu thao tác DOM thật.

Bài toán trừu tượng hóa thành:

“Cho hai cây (hoặc hai danh sách có thứ tự) A và B dài  $n$ , tìm dãy thao tác **chèn-xóa-di chuyển** tối ưu biến A thành B”.

### 2. Độ phức tạp $O(n^3)$ – phiên bản “chuẩn hóa” của lý thuyết

Bài toán gốc tương đương “**string edit distance với phép move**” hoặc “**tree-to-tree edit distance**”:

1. Với **chuỗi** cho phép “insert / delete / replace”, thuật toán *dynamic programming* kinh điển (Levenshtein) chạy  $O(n^2)$ .
2. Thêm **move** (đổi vị trí phần tử) → độ phức tạp nhảy vọt:
  - Shasha & Zhang tree-edit distance (1989) cho hai cây tổng  $n$  node:  $O(n^3)$  thời gian,  $O(n^2)$  bộ nhớ.

→ Nếu React/Vue cố gắng kiểm **phương án tối ưu toàn cục** (ít thao tác DOM nhất) thì tệ nhất phải đi toàn bộ ma trận 3 chiều (node cũ  $\times$  node mới  $\times$  depth)  $\Rightarrow O(n^3)$ .

### 3. Ý tưởng then chốt để giảm xuống $O(n)$

Framework “thông minh” nhưng **không theo đuổi tuyệt đối tối ưu**; thay vào đó họ đặt **giả định thực tế**:

| Giả định (Heuristic)                                                                                                                    | Hậu quả                              | Kết quả độ phức tạp            |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|--------------------------------|
| 1. <b>Cùng cấp không cross-level</b> : Chỉ so sánh node <b>cùng depth</b> ; nếu kiểu khác $\Rightarrow$ thay thế toàn subtree           | Bỏ hẳn việc so depth-mix             | loại một chiều lặp             |
| 2. <b>Key ổn định</b> : Khi list có key, coi việc “move” là phát hiện thông qua key $\rightarrow$ dùng <b>Map tra <math>O(1)</math></b> | Không cần tìm đường đi chuyển tối ưu | tuyến tính theo chiều dài list |
| 3. <b>Sequential scan</b> : Nếu phần lớn phần tử <b>không đảo lộn cực đoan</b> , chỉ cần hai con trỏ trái/phải quét                     | Giống thuật toán LCS rút gọn         | $O(n)$                         |

#### 3.1. Diff của React (Fiber 16+)

1. **Bước 1 – so type**: khác type  $\Rightarrow$  unmount + mount ( $O(1)$ ).
2. **Bước 2 – key list reconcile**:
  - Lần 1 quét nối tiếp  $\rightarrow$  clone/sửa node trùng vị trí ( $O(n)$ ).
  - Tìm thấy “lệch” đầu tiên  $\rightarrow$  xây Map <key, index01d> rồi quét lần 2 ( $O(n)$ ).
  - Move sử dụng chỉ số cũ/ mới  $\rightarrow$  không cần thuật toán tối ưu toàn cục.
3. Không xử lý cross-level, vì thế bỏ được nhân tố  $n^2$  cho depth  $\rightarrow$   **$O(n)$** .

#### 3.2. Diff của Vue 3 (基于 React 思路)

- Áp dụng “双端指针” (head-tail pointers): so sánh 4 trường hợp (head-head, tail-tail, head-tail, tail-head).
- Khi rơi vào “đoạn giữa lộn xộn”, chạy **最长递增子序列 (LIS)** trên vị trí mới để tính minimal move.
  - LIS dùng binary search  $O(k \log k)$  ( $k \leq n$ ) nhưng phần còn lại vẫn tuyến tính, trung bình **gần  $O(n)$** ; tệ nhất  $O(n \log n)$ .

### 4. Tóm lược cách “tính” hai độ phức tạp

1.  **$O(n^3)$**  = kết quả học thuật khi:
  - So sánh *mọi* node cũ với *mọi* node mới ở *mọi* nhánh
  - Phải tìm edit sequence tối ưu toàn cục (tree edit distance).
2.  **$O(n)$**  (hoặc  $n \log n$ ) = khi:
  - Giới hạn so sánh cùng depth, bỏ phép “đổi cấp”.
  - Dựa vào key/position để biến việc di chuyển thành tra cứu map + quét tuần tự.
  - Một pass (hoặc hai pass) trên danh sách con  $\Rightarrow$  thời gian tỉ lệ trực tiếp với tổng node.

## 5. Ví dụ “dummy” minh họa

Giả sử list cũ: [A, B, C, D]

List mới: [A, C, B, E]

| Thuật toán                | Bước quét                                                                                                                  | Thao tác tìm được                 |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Edit distance<br>$O(n^3)$ | Duyệt mọi cặp, fill bảng                                                                                                   | 1.swap B,C<br>2.replace D → E     |
| React/Vue<br>$O(n)$       | Quét trái → phải: ①A match; ②B ≠ C → lưu map cũ, tiếp tục; ③C tồn tại → move trước B; ④B ở map → move sau C; hết → mount E | 2 move + 1 insert<br>(tuyến tính) |

Mặc dù thao tác không tối ưu “ít nhất tuyệt đối”, chi phí tính toán **giảm từ hàng triệu bước xuống vài nghìn** khi n lớn, tốc độ thực tế **vượt trội** và UI vẫn chính xác.

## 6. Kết luận

- $O(n^3)$  phát sinh từ mô hình so khớp tổng quát nhất cho cây DOM.
- React/Vue **chấp nhận đánh đổi**: đặt các *heuristic thực tế* (cùng cấp, key, quét hai đầu) → biến diff thành **một/hai lượt duyệt** trên chiều dài danh sách, nên  $O(n)$  (hoặc  $n \log n$  với phép LIS).
- Đây là ví dụ điển hình “giảm độ phức tạp bằng giả định domain-specific” – triết lý thiết kế thuật toán thường dùng trong kỹ thuật front-end hiệu năng cao.

**Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt):**

第 98 题：（京东）写出如下代码的打印结果

js

代码解读

复制代码

```
function changeObjProperty(o) {
 o.siteUrl = "http://www.baidu.com"
 o = new Object()
 o.siteUrl = "http://www.google.com"
}

let webSite = new Object();
changeObjProperty(webSite);
console.log(webSite.siteUrl);
```

Khi chạy đoạn code sau:

```
function changeObjProperty(o) {
 o.siteUrl = "http://www.baidu.com";
```

```
o = new Object();
o.siteUrl = "http://www.google.com";
}

let webSite = new Object();
changeObjProperty(webSite);
console.log(webSite.siteUrl);
```

Kết quả in ra sẽ là:

```
http://www.baidu.com
```

#### — Giải thích chi tiết:

1. Khi gọi `changeObjProperty(webSite)`, tham số `o` nhận **tham chiếu** tới đối tượng `webSite`.
2. Dòng `o.siteUrl = "http://www.baidu.com"` sẽ thêm (hay gán) thuộc tính `siteUrl` lên chính đối tượng `webSite`.
3. Tiếp theo, `o = new Object()` sẽ khiến biến cục bộ `o` trở tới một đối tượng mới hoàn toàn, **không** còn trở tới `webSite` nữa.
4. Việc gán `o.siteUrl = "http://www.google.com"` chỉ ảnh hưởng tới đối tượng mới này, **không** đụng chạm đến `webSite`.
5. Khi kết thúc hàm, `webSite` vẫn giữ nguyên `siteUrl = "http://www.baidu.com"`.

#### — Bản chất concept:

- **Truyền tham chiếu (pass-by-value của tham chiếu):** Trong JavaScript, khi truyền một object vào hàm, bạn truyền **bản sao của tham chiếu** đó. Mọi sửa đổi trên thuộc tính của object qua tham chiếu này sẽ ảnh hưởng tới nguyên bản.
- **Gán lại biến tham chiếu** (`o = ...`) chỉ thay đổi tham chiếu cục bộ bên trong hàm, không thể "thay đổi" biến gốc ở ngoài hàm.

#### — Ví dụ dummy mở rộng:

```
function foo(obj) {
 obj.name = 'Alice'; // 1. thay đổi thuộc tính
 obj = { name: 'Bob' }; // 2. gán lại tham chiếu cục bộ
 obj.name = 'Charlie'; // 3. thay đổi đối tượng mới
}

const user = { name: 'Eve' };
foo(user);
console.log(user.name); // 'Alice' (không phải 'Charlie')
```

#### — Kết luận nâng cao:

- Khi muốn trong hàm **thay hoàn toàn** một object ngoài, bạn cần trả về object mới và gán lại ở bên ngoài:

```
function replace(o) {
 return { siteUrl: "http://www.google.com" };
}
webSite = replace(webSite);
```

- Hoặc dùng **mutate** đối tượng ban đầu thay vì gán lại biến tham chiếu:

```
function mutate(o) {
 Object.assign(o, { siteUrl: "http://www.google.com" });
}
mutate(webSite);
console.log(webSite.siteUrl); // now google.com
```

## BÀI 99 – "REVERSE INTEGER TO STRING" với Độ quy Pure

### I. PHÂN TÍCH BÀI TOÁN

**Đầu vào:** Số nguyên (int)

**Đầu ra:** Chuỗi (string) đã đảo ngược

**Constraints:**

- Chỉ được dùng đệ quy (recursive)
- Không global variables
- Chỉ một parameter
- Phải return string

**Thách thức:** Với chỉ một parameter, làm sao track state trong đệ quy?

### II. CÁC PHƯƠNG PHÁP TIẾP CẬN

#### 1. String-based Recursion (Đơn giản nhất)

```
function reverseInt(n) {
 // Convert to string first, then recursive on string
 const str = String(Math.abs(n)); // handle negative

 function reverseStr(s) {
 // Base case: empty or single char
 if (s.length <= 1) return s;

 // Recursive: last char + reverse(remaining)
 return s[s.length - 1] + reverseStr(s.slice(0, -1));
 }

 const result = reverseStr(str);
 return n < 0 ? '-' + result : result;
}
```

```
// Test
console.log(reverseInt(1234)); // "4321"
console.log(reverseInt(-567)); // "-765"
console.log(reverseInt(0)); // "0"
```

**Ưu điểm:** Đơn giản, dễ hiểu

**Nhược điểm:** Tạo nhiều substring →  $O(n^2)$  space complexity

## 2. Mathematical Approach (Thuật toán số học thuần)

```
function reverseInt(n) {
 const sign = n < 0 ? '-' : '';
 const abs = Math.abs(n);

 function extractAndReverse(num) {
 // Base case: single digit
 if (num < 10) return String(num);

 // Extract last digit and recurse on remaining
 const lastDigit = num % 10;
 const remaining = Math.floor(num / 10);

 return String(lastDigit) + extractAndReverse(remaining);
 }

 return abs === 0 ? '0' : sign + extractAndReverse(abs);
}

console.log(reverseInt(1234)); // "4321"
console.log(reverseInt(-567)); // "-765"
```

**Ưu điểm:**  $O(\log n)$  space, thuần toán học

**Nhược điểm:** Phức tạp với số 0, âm

## 3. Closure với Helper Function (Recommended)

```
function reverseInt(n) {
 const sign = n < 0 ? '-' : '';
 const num = Math.abs(n);

 // Inner recursive function với closure
 function helper(remaining, accumulated = '') {
 // Base case: no more digits
 if (remaining === 0) return accumulated || '0';

 // Extract last digit and recurse
 const digit = remaining % 10;
 const next = Math.floor(remaining / 10);

 return helper(next, accumulated + String(digit));
 }
}
```

```
 return sign + helper(num);
 }
```

**Ưu điểm:** Tail recursion friendly, clean separation of concerns

### 4. One-liner với Template Recursion

```
function reverseInt(n) {
 return n < 0
 ? '-' + reverseInt(-n)
 : n < 10
 ? String(n)
 : String(n % 10) + reverseInt(Math.floor(n / 10));
}

console.log(reverseInt(1234)); // "4321"
console.log(reverseInt(-567)); // "-765"
```

**Ưu điểm:** Compact, elegant  
**Nhược điểm:** Khó debug, stack overflow với số lớn

## III. PERFORMANCE ANALYSIS

| Approach       | Time        | Space       | Readability | Edge Case Handling |
|----------------|-------------|-------------|-------------|--------------------|
| String-based   | $O(n^2)$    | $O(n^2)$    | *****       | ***                |
| Mathematical   | $O(\log n)$ | $O(\log n)$ | ***         | **                 |
| Closure Helper | $O(\log n)$ | $O(\log n)$ | *****       | *****              |
| One-liner      | $O(\log n)$ | $O(\log n)$ | **          | *****              |

## IV. EDGE CASES VÀ ROBUST VERSION

```
function reverseIntRobust(n) {
 // Input validation
 if (!Number.isInteger(n)) {
 throw new TypeError('Input must be an integer');
 }

 // Handle overflow (JavaScript safe integer range)
 if (Math.abs(n) > Number.MAX_SAFE_INTEGER) {
 throw new RangeError('Input exceeds safe integer range');
 }

 const sign = n < 0 ? '-' : '';
 const num = Math.abs(n);

 function reverseDigits(remaining, result = '') {
 if (remaining === 0) {
 return result || '0'; // Handle zero case
 }
 }
```

```

 const lastDigit = remaining % 10;
 const nextNum = Math.floor(remaining / 10);

 return reverseDigits(nextNum, result + String(lastDigit));
 }

 return sign + reverseDigits(num);
}

// Comprehensive test suite
const testCases = [
 { input: 1234, expected: '4321' },
 { input: -567, expected: '-765' },
 { input: 0, expected: '0' },
 { input: 1, expected: '1' },
 { input: -1, expected: '-1' },
 { input: 1000, expected: '0001' },
 { input: -1000, expected: '-0001' },
 { input: 123456789, expected: '987654321' }
];

testCases.forEach(({ input, expected }, i) => {
 try {
 const result = reverseIntRobust(input);
 console.log(`Test ${i + 1}: ${result} === expected ? '✓' : '✗'`);
 Input: ${input}, Expected: "${expected}", Got: "${result}"`);
 } catch (error) {
 console.log(`Test ${i + 1}: ✗ Error: ${error.message}`);
 }
});

```

## V. OPTIMIZATION TECHNIQUES

### 5.1. Tail Call Optimization (TCO)

```

// TCO-friendly version (ES6+)
function reverseIntTCO(n) {
 'use strict'; // Enable TCO in supporting engines

 const sign = n < 0 ? '-' : '';
 const num = Math.abs(n);

 // Tail recursive helper
 function tailReverse(remaining, accumulator = '') {
 if (remaining === 0) {
 return accumulator || '0';
 }

 // Tail call - last operation is function call
 return tailReverse(
 Math.floor(remaining / 10),
 accumulator + String(remaining % 10)
);
 }
}

```

```

 }

 return sign + tailReverse(num);
}

```

## 5.2. Memoization cho Multiple Calls

```

function createReverseInt() {
 const cache = new Map();

 function reverseInt(n) {
 if (cache.has(n)) {
 return cache.get(n);
 }

 const sign = n < 0 ? '-' : '';
 const num = Math.abs(n);

 function helper(remaining, result = '') {
 if (remaining === 0) return result || '0';

 const digit = remaining % 10;
 const next = Math.floor(remaining / 10);
 return helper(next, result + String(digit));
 }

 const result = sign + helper(num);
 cache.set(n, result);
 return result;
 }

 // Add cache management methods
 reverseInt.clearCache = () => cache.clear();
 reverseInt.getCacheSize = () => cache.size;

 return reverseInt;
}

// Usage
const reverseInt = createReverseInt();
console.log(reverseInt(1234)); // First call: compute and cache
console.log(reverseInt(1234)); // Second call: return from cache

```

## VI. BIG INTEGER SUPPORT

```

function reverseIntBig(n) {
 // Handle BigInt or regular numbers
 const isBigInt = typeof n === 'bigint';
 const str = String(n);
 const isNegative = str.startsWith('-');
 const digits = isNegative ? str.slice(1) : str;

 function reverseDigits(s) {

```

```

 if (s.length <= 1) return s;
 return s[s.length - 1] + reverseDigits(s.slice(0, -1));
 }

 const reversed = reverseDigits(digits);
 return (isNegative ? '-' : '') + reversed;
}

// Test with BigInt
console.log(reverseIntBig(12345678901234567890n)); // "09876543210987654321"
console.log(reverseIntBig(-12345678901234567890n)); // "-09876543210987654321"

```

## VII. FUNCTIONAL PROGRAMMING APPROACH

```

// Purely functional with higher-order functions
const reverseInt = (n) => {
 const compose = (f, g) => x => f(g(x));
 const curry = fn => (...args) => args.length >= fn.length
 ? fn(...args)
 : (...nextArgs) => curry(fn)(...args, ...nextArgs);

 const abs = Math.abs;
 const sign = n => n < 0 ? '-' : '';
 const toString = x => String(x);

 const reverseRecursive = curry((acc, num) =>
 num === 0
 ? acc || '0'
 : reverseRecursive(acc + String(num % 10), Math.floor(num / 10))
);

 return sign(n) + reverseRecursive('')(abs(n));
};

```

## VIII. PERFORMANCE BENCHMARKING

```

function benchmark() {
 const methods = {
 'String-based': reverseIntString,
 'Mathematical': reverseInt,
 'TCO': reverseIntTCO,
 'Memoized': createReverseInt()
 };

 const testInput = 123456789;
 const iterations = 100000;

 Object.entries(methods).forEach(([name, fn]) => {
 console.time(name);
 for (let i = 0; i < iterations; i++) {
 fn(testInput);
 }
 console.timeEnd(name);
 });
}

```

```
});
}

// Uncomment to run benchmark
// benchmark();
```

## IX. REAL-WORLD APPLICATIONS

### 9.1. Palindrome Number Checker

```
function isPalindromeNumber(n) {
 if (n < 0) return false;
 const str = String(n);
 const reversed = reverseInt(n);
 return str === reversed;
}

console.log(isPalindromeNumber(12321)); // true
console.log(isPalindromeNumber(12345)); // false
```

### 9.2. Cryptographic Applications

```
function simpleNumberObfuscation(n) {
 // Simple obfuscation by reversing digits
 const reversed = reverseInt(n);
 const numeric = parseInt(reversed);
 return numeric ^ 0x12345; // XOR with constant
}

function deobfuscate(obfuscated) {
 const xored = obfuscated ^ 0x12345;
 return parseInt(reverseInt(xored));
}

const original = 123456;
const obfuscated = simpleNumberObfuscation(original);
const restored = deobfuscate(obfuscated);
console.log({ original, obfuscated, restored });
```

### 9.3. Data Processing Pipeline

```
// Process array of integers with reverse operation
function processIntegerArray(arr) {
 return arr
 .filter(n => Number.isInteger(n))
 .map(reverseInt)
 .filter(str => !str.includes('0')) // Remove reversed with leading zeros
 .sort((a, b) => a.length - b.length || a.localeCompare(b));
}
```

```
const data = [1234, 5670, 890, -123, 0, 42];
console.log(processIntegerArray(data));
// ["24", "321", "-321", "4321"]
```

## X. DEBUGGING VÀ VISUALIZATION

```
function reverseIntWithTrace(n) {
 const trace = [];

 function helper(remaining, result = '', depth = 0) {
 const indent = ' '.repeat(depth);
 trace.push(`${indent}Call: remaining=${remaining}, result="${result}"`);

 if (remaining === 0) {
 const final = result || '0';
 trace.push(`${indent}Return: "${final}"`);
 return final;
 }

 const digit = remaining % 10;
 const next = Math.floor(remaining / 10);
 const newResult = result + String(digit);

 const outcome = helper(next, newResult, depth + 1);
 trace.push(`${indent}Return: "${outcome}"`);
 return outcome;
 }

 const sign = n < 0 ? '-' : '';
 const result = sign + helper(Math.abs(n));

 console.log('Execution trace:');
 trace.forEach(line => console.log(line));

 return result;
}

// Visualize execution
console.log('Final result:', reverseIntWithTrace(1234));
```

## XI. TYPESCRIPT VERSION

```
function reverseInt(n: number): string {
 if (!Number.isInteger(n)) {
 throw new TypeError('Input must be an integer');
 }

 const sign: string = n < 0 ? '-' : '';
 const num: number = Math.abs(n);

 function helper(remaining: number, accumulated: string = ''): string {
 if (remaining === 0) {
 return accumulated || '0';
 }
 }
```

```

 }

 const digit: number = remaining % 10;
 const next: number = Math.floor(remaining / 10);

 return helper(next, accumulated + String(digit));
 }

 return sign + helper(num);
}

// Type-safe usage
const result: string = reverseInt(1234);
console.log(result); // "4321"

```

## XII. KẾT LUẬN VÀ BEST PRACTICES

### 12.1. Recommended Solution

Để production code, khuyến nghị dùng **Closure Helper approach**:

```

function reverseInt(n) {
 const sign = n < 0 ? '-' : '';
 const num = Math.abs(n);

 function helper(remaining, result = '') {
 if (remaining === 0) return result || '0';

 const digit = remaining % 10;
 const next = Math.floor(remaining / 10);

 return helper(next, result + String(digit));
 }

 return sign + helper(num);
}

```

### 12.2. Key Takeaways

1. **Constraint-driven design** dẫn đến creative solutions
2. **Recursive thinking** khác với iterative - focus on base cases
3. **Edge case handling** critical cho production code
4. **Performance considerations** quan trọng với large inputs
5. **Testing comprehensively** đảm bảo correctness

## 12.3. Common Pitfalls

- Quên xử lý số âm
- Không handle số 0 đúng cách
- Stack overflow với số quá lớn
- Không validate input type
- Performance degradation với repeated calls

**Pattern này** hữu ích cho:

- String manipulation algorithms
- Mathematical computation recursion
- Tree/graph traversal problems
- Divide-and-conquer approaches

Hiểu sâu "single-parameter recursive design" giúp solve các bài toán constraint tương tự và thiết kế clean recursive solutions trong real-world applications.

## BÀI 100 – "FUNCTION PROPERTY & PROTOTYPE CHAIN" – Phân tích Method Resolution

### I. PHÂN TÍCH EXECUTION FLOW

Đây là bài kiểm tra hiểu biết về **static methods**, **instance methods**, **prototype chain** và **method overriding** trong JavaScript.

### II. TRACE EXECUTION STEP-BY-STEP

```
function Foo() {
 Foo.a = function() { // ① Override static method
 console.log(1)
 }
 this.a = function() { // ② Set instance method
 console.log(2)
 }
}

Foo.prototype.a = function() { // ③ Prototype method
 console.log(3)
}

Foo.a = function() { // ④ Initial static method
 console.log(4)
}

// === EXECUTION ===
Foo.a(); // ⑤ Call static method
let obj = new Foo(); // ⑥ Create instance (runs constructor)
```

```
obj.a(); // ⑦ Call instance method
Foo.a(); // ⑧ Call static method again
```

### III. KẾT QUẢ THỰC THI

Output:

```
4
2
1
```

### IV. GIẢI THÍCH CHI TIẾT

**Bước 1:** `Foo.a()` - Lần gọi đầu tiên

```
// Tại thời điểm này: Foo.a = function() { console.log(4) }
Foo.a(); // → 4
```

**Bước 2:** `let obj = new Foo()` - Tạo instance

```
// Constructor được chạy:
// 1. Foo.a = function() { console.log(1) } // Override static method
// 2. this.a = function() { console.log(2) } // Set instance property

// Sau khi constructor chạy xong:
// - Foo.a đã bị override → function() { console.log(1) }
// - obj.a được set → function() { console.log(2) }
```

**Bước 3:** `obj.a()` - Gọi method trên instance

```
// Property lookup order:
// 1. obj own property ✓ → obj.a = function() { console.log(2) }
// 2. Foo.prototype.a (not reached)

obj.a(); // → 2
```

**Bước 4:** `Foo.a()` - Lần gọi thứ hai

```
// Foo.a đã bị override trong constructor
// Foo.a = function() { console.log(1) }
Foo.a(); // → 1
```

## V. VISUALIZATION CỦA METHOD RESOLUTION

```
// Memory structure after `new Foo()`

Foo (Function Object)
├── a: function() { console.log(1) } // Static method (overridden)
└── prototype
 └── a: function() { console.log(3) } // Prototype method

obj (Instance)
├── a: function() { console.log(2) } // Instance method (own property)
├── __proto__ → Foo.prototype
└── constructor → Foo
```

## VI. PROPERTY LOOKUP CHAIN

```
// When calling obj.a():
// 1. Check obj.hasOwnProperty('a') → TRUE ✓
// Return: function() { console.log(2) }
//
// 2. (Not reached) Check obj.__proto__.a (Foo.prototype.a)
// Would return: function() { console.log(3) }
```

## VII. CONCEPT DEMONSTRATIONS

### 7.1. Static vs Instance vs Prototype Methods

```
function Demo() {
 // Instance method (own property)
 this.method = function() { return 'instance'; };
}

// Prototype method
Demo.prototype.method = function() { return 'prototype'; };

// Static method
Demo.method = function() { return 'static'; };

console.log(Demo.method()); // "static"
const d = new Demo();
console.log(d.method()); // "instance" (shadows prototype)
console.log(Demo.method()); // "static" (unchanged)

// Remove instance method to reveal prototype
delete d.method;
console.log(d.method()); // "prototype"
```

## 7.2. Method Shadowing Hierarchy

```
function TestShadowing() {
 this.getValue = () => 'instance';
}

TestShadowing.prototype.getValue = () => 'prototype';
TestShadowing.getValue = () => 'static';

const test = new TestShadowing();

// Priority order:
console.log(test.getValue()); // "instance"
console.log(test.__proto__.getValue()); // "prototype"
console.log(test.constructor.getValue()); // "static"
```

## 7.3. Dynamic Method Override

```
function DynamicFoo() {
 // Override static method during construction
 DynamicFoo.count = (DynamicFoo.count || 0) + 1;

 if (DynamicFoo.count === 1) {
 DynamicFoo.getType = () => 'first instance created';
 } else {
 DynamicFoo.getType = () => 'multiple instances';
 }

 this.getInstanceId = () => DynamicFoo.count;
}

DynamicFoo.getType = () => 'no instances yet';

console.log(DynamicFoo.getType()); // "no instances yet"
const obj1 = new DynamicFoo();
console.log(DynamicFoo.getType()); // "first instance created"
const obj2 = new DynamicFoo();
console.log(DynamicFoo.getType()); // "multiple instances"
```

## VIII. EDGE CASES VÀ GOTCHAS

### 8.1. Arrow Functions vs Regular Functions

```
function ArrowTest() {
 // Arrow function - binds 'this' lexically
 this.arrow = () => console.log('arrow:', this.constructor.name);

 // Regular function - 'this' determined by call site
 this.regular = function() { console.log('regular:', this.constructor.name); };
}
```

```

const at = new ArrowTest();
const extracted = at.arrow;
const extractedRegular = at.regular;

at.arrow(); // "arrow: ArrowTest"
extracted(); // "arrow: ArrowTest" (still bound)
at.regular(); // "regular: ArrowTest"
extractedRegular(); // "regular: undefined" (lost context)

```

## 8.2. Prototype Pollution Risk

```

function Vulnerable() {
 // Dangerous: modifying prototype during construction
 if (Math.random() > 0.5) {
 Vulnerable.prototype.exploit = function() {
 console.log('All instances affected!');
 };
 }
}

const v1 = new Vulnerable();
const v2 = new Vulnerable();

// Both instances might have the exploit method
if (v1.exploit) v1.exploit();
if (v2.exploit) v2.exploit();

```

## 8.3. Memory Implications

```

function MemoryTest() {
 // BAD: Creates new function for each instance
 this.inefficient = function() { return 'unique per instance'; };
}

// GOOD: Shared across all instances
MemoryTest.prototype.efficient = function() { return 'shared'; };

// Create 1000 instances
const instances = Array.from({length: 1000}, () => new MemoryTest());

// Each instance has its own 'inefficient' function
// All instances share the same 'efficient' function

```

## IX. REAL-WORLD PATTERNS

## 9.1. Singleton Pattern với Static Override

```
function Singleton() {
 if (Singleton.instance) {
 return Singleton.instance;
 }

 // Override constructor để trả về instance duy nhất
 Singleton = function() {
 return Singleton.instance;
 };

 // Preserve prototype
 Singleton.prototype = this.constructor.prototype;

 Singleton.instance = this;
 this.created = new Date();
}

const s1 = new Singleton();
const s2 = new Singleton();
console.log(s1 === s2); // true
```

## 9.2. Factory Pattern với Method Switching

```
function ShapeFactory(type) {
 // Switch static method based on usage
 if (!ShapeFactory.createCount) ShapeFactory.createCount = 0;
 ShapeFactory.createCount++;

 if (ShapeFactory.createCount > 10) {
 ShapeFactory.create = function() {
 throw new Error('Factory limit exceeded');
 };
 }

 this.type = type;
 this.area = ShapeFactory.calculateArea[type] || (() => 0);
}

ShapeFactory.calculateArea = {
 circle: function() { return Math.PI * this.radius ** 2; },
 square: function() { return this.side ** 2; }
};

ShapeFactory.create = function(type) {
 return new ShapeFactory(type);
};
```

### 9.3. Plugin System với Dynamic Methods

```
function PluginHost() {
 this.plugins = [];

 // Dynamic method installation
 this.installPlugin = function(name, implementation) {
 this[name] = implementation.bind(this);
 this.plugins.push(name);

 // Update static registry
 PluginHost.installedPlugins = PluginHost.installedPlugins || [];
 PluginHost.installedPlugins.push(name);
 };
}

const host = new PluginHost();
host.installPlugin('logger', function(msg) {
 console.log(`[${new Date().toISOString()}] ${msg}`);
});

host.logger('Plugin system active'); // Uses dynamically added method
```

## X. DEBUGGING TECHNIQUES

### 10.1. Method Resolution Tracer

```
function traceMethodResolution(obj, methodName) {
 const chain = [];
 let current = obj;

 while (current) {
 if (current.hasOwnProperty(methodName)) {
 chain.push({
 level: current === obj ? 'own' : 'inherited',
 object: current.constructor.name || 'Object',
 method: current[methodName]
 });
 break;
 }

 if (current[methodName]) {
 chain.push({
 level: 'inherited',
 object: current.constructor.name || 'Object',
 method: current[methodName]
 });
 break;
 }

 current = Object.getPrototypeOf(current);
 }
}
```

```

 return chain;
}

// Usage with our original example
function Foo() {
 this.a = function() { console.log(2); };
}
Foo.prototype.a = function() { console.log(3); };

const obj = new Foo();
console.table(traceMethodResolution(obj, 'a'));

```

## XI. PERFORMANCE CONSIDERATIONS

```

// Benchmark: Instance vs Prototype methods
function BenchmarkTest() {
 this.instanceMethod = function() { return 'instance'; };
}

BenchmarkTest.prototype.prototypeMethod = function() { return 'prototype'; };

const instances = Array.from({length: 10000}, () => new BenchmarkTest());

console.time('Instance methods');
instances.forEach(obj => obj.instanceMethod());
console.timeEnd('Instance methods');

console.time('Prototype methods');
instances.forEach(obj => obj.prototypeMethod());
console.timeEnd('Prototype methods');

// Prototype methods are typically faster (shared reference)
// Instance methods use more memory (unique per instance)

```

## XII. TYPESCRIPT VERSION

```

class TypeScriptFoo {
 // Static method
 static a(): void {
 console.log(4);
 }

 // Instance method (will be overridden)
 a(): void {
 console.log(3);
 }

 constructor() {
 // Override static method
 TypeScriptFoo.a = () => console.log(1);

 // Override instance method
 this.a = () => console.log(2);
 }
}

```

```

 }
}

// Same behavior, but with type safety
TypeScriptFoo.a(); // 4
const obj = new TypeScriptFoo(); // Constructor runs
obj.a(); // 2
TypeScriptFoo.a(); // 1

```

## XIII. KẾT LUẬN VÀ BEST PRACTICES

### 13.1. Key Takeaways

1. **Method Resolution Order:** Own property → Prototype chain → undefined
2. **Static vs Instance:** Static methods belong to constructor, instance to objects
3. **Constructor Side Effects:** Constructors can modify static properties
4. **Memory Trade-offs:** Instance methods use more memory but can be customized

### 13.2. Best Practices

```

// ✓ GOOD: Use prototype for shared methods
function GoodExample() {
 this.id = Math.random();
}
GoodExample.prototype.getId = function() { return this.id; };

// ✗ BAD: Create methods in constructor
function BadExample() {
 this.id = Math.random();
 this.getId = function() { return this.id; }; // New function per instance
}

// ✓ GOOD: Static methods for utilities
GoodExample.isValid = function(obj) {
 return obj instanceof GoodExample && obj.id;
};

// ✗ BAD: Modifying static properties in constructor
function BadExample() {
 BadExample.count = (BadExample.count || 0) + 1; // Side effect
}

```

### 13.3. Common Pitfalls

- Confusing static vs instance method calls
- Creating unnecessary functions in constructors
- Not understanding prototype chain shadowing
- Memory leaks from constructor side effects

**Pattern này** quan trọng cho:

- Object-oriented design patterns
- Plugin architectures
- Factory patterns
- Framework internals understanding

Hiểu sâu "method resolution và property shadowing" giúp debug JS objects effectively và design clean OOP architectures trong large-scale applications.

**Bạn là 1 Principal Engineer tại Google, giải thích chi tiết chuyên sâu khái niệm, bản chất các concept trên và mở rộng nâng cao liên quan bao gồm ví dụ cho dummy (tiếng việt):**

第 100 题：（京东）请写出如下代码的打印结果

js

代码解读

复制代码

```
function Foo() {
 Foo.a = function() {
 console.log(1)
 }
 this.a = function() {
 console.log(2)
 }
}
Foo.prototype.a = function() {
 console.log(3)
}
Foo.a = function() {
 console.log(4)
}
Foo.a();
let obj = new Foo();
obj.a();
Foo.a();
```

Kết quả đoạn code in ra sẽ là:

```
4
2
1
```

## 1. Khai báo ban đầu

```
function Foo() { ... }
Foo.prototype.a = function(){ console.log(3) };
Foo.a = function(){ console.log(4) };
```

- Foo lúc này là một *function object* có:
- một phương thức tính `Foo.a()` in ra 4
- trên `Foo.prototype` có phương thức `a()` in ra 3
- chưa có `this.a` nào trên instance.

## 2. Gọi `Foo.a()` lần đầu

```
Foo.a(); // in 4
```

- Vì Foo có *own property* a ban đầu (đã gán `console.log(4)`), nên gọi thẳng in 4.

## 3. Tạo instance `let obj = new Foo();`

Khi khởi tạo:

```
function Foo(){
 // → Gán lại phương thức tính
 Foo.a = function(){ console.log(1) };
 // → Gán phương thức riêng cho instance
 this.a = function(){ console.log(2) };
}
```

- `Foo.a = ...` sẽ thay thế method tính trước đó, giờ `Foo.a()` in ra 1.
- `this.a = ...` tạo *own property* a trên `obj`, in ra 2.
- `Foo.prototype.a` vẫn còn nhưng bị *shadow* bởi `obj.a`.

## 4. Gọi `obj.a()`

```
obj.a(); // in 2
```

- Khi gọi `obj.a` JS tìm theo thứ tự:
  1. Own property trên `obj` → thấy `function(){ console.log(2) }` → in 2.
  2. (Nếu không có) mới tới `Foo.prototype.a` in 3.

## 5. Gọi `Foo.a()` lần hai

```
Foo.a(); // in 1
```

- Do constructor đã gán lại `Foo.a = ...` thành function in 1, nên lần này in 1.

— Bản chất và khái niệm liên quan

- JavaScript không có "static method" riêng biệt: function object cũng là object, bạn chỉ việc gán property lên đó (`Foo.a = ...`).
- **Own property** trên instance (`this.a`) luôn thắng **prototype property** (`Foo.prototype.a`).

- Gán lại biến tham chiếu cục bộ (`o = ...`) khác với *mutate* object gốc (`o.x = ...`), chỉ *mutate* làm ảnh hưởng bên ngoài.
- Khi khởi tạo với `new Foo()`:
- Tạo object rỗng, gán `__proto__ = Foo.prototype`.
- Chạy hàm `Foo` với `this` trở vào instance mới.

— Mô hình bộ nhớ (sau `new Foo()`)

```

Foo (Function)
├─ a: f(){ console.log(1) } // static method (đã override)
└─ prototype → Foo.prototype
 └─ a: f(){ console.log(3) } // prototype method

obj (Instance)
├─ a: f(){ console.log(2) } // own property → instance method
└─ __proto__ → Foo.prototype

```

— Mở rộng nâng cao

### 1. Dynamic Override / Singleton Pattern

Bạn có thể dùng constructor để “bật” chế độ một lần:

```

function Config(){
 if (!Config.initialized){
 Config.get = () => {/*...*/};
 Config.initialized = true;
 }
}

```

### 2. Performance & Memory

- Tạo method trong constructor (`this.a = ...`) sẽ sinh ra *function object* riêng cho mỗi instance → tốn bộ nhớ nếu tạo nhiều.
- Nên đặt các method chung lên `prototype` để chia sẻ code giữa các instance, giảm footprint.

### 3. Arrow Function vs. Function Expression

- Nếu bạn gán `this.a = () => console.log(2)` thì arrow sẽ *bind* lexical `this`, giữ luôn context; ngược lại `function(){...}` có thể bị thay đổi `this` khi tách ra gọi.

### 4. Gotchas / Pitfalls

- Tránh *prototype pollution* (động gán lên `Foo.prototype` từ bên ngoài) nếu dữ liệu đầu vào không tin cậy.
- Cảnh trọng khi constructor thay đổi hần bản đồ static, dễ gây hiểu nhầm cho người đọc.

### 5. Ví dụ dummy minh họa “method shadowing” tương tự

```

function Widget(name){
 // Override static factory
 Widget.create = () => console.log('Factory disabled');
}

```

```
// Gán instance method
this.describe = () => console.log('Instance of', name);
}
// Prototype method
Widget.prototype.describe = function(){
 console.log('Prototype widget');
};
// Static khởi tạo
Widget.create = () => console.log('Create new widget');

Widget.create(); // Create new widget
const w = new Widget('MyWidget');
w.describe(); // Instance of MyWidget
Widget.create(); // Factory disabled
```

## — Kết luận

- Hiểu rõ **đối tượng Function** là object với prototype và có thể gán property tĩnh giúp bạn khai thác pattern như factory, singleton.
- Nắm vững thứ tự lookup (own property → prototype → prototype của prototype ...) để debug và thiết kế class/function đúng chuẩn.
- Biết trade-off giữa *instance method* (tùy biến, tốn nhớ) và *prototype method* (dùng chung, tiết kiệm).

1. [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Promise/race](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/race)
2. <https://v5.chriskrycho.com/notes/javascript-promise-race-and-promise-all-are-not-fair/>